

PROMPT TO PROFIT™

WORKBOOK 04

Professional Invoice Generator

Build a Complete Software Application with AI

TECHNOLOGY STACK

HTML

CSS

Vanilla JavaScript

Supabase

DIFFICULTY

Beginner

ESTIMATED COMPLETION TIME

25-30 Hours

PRODUCED BY

Tochukwu Tech and AI Academy

www.tochukwunkwocha.com

PROMPT TO PROFIT™

Workbook 04 · Professional Invoice Generator

© 2026 Tochukwu Tech and AI Academy. All rights reserved.

Produced and published by Tochukwu Tech and AI Academy.

www.tochukwunkwocha.com

This workbook is designed for personal learning and guided software-building practice.

Technology names and product names remain the property of their respective owners.

Contents

Workbook 04 · Professional Invoice Generator

LEARNER SUPPORT TOOLKIT	6
VISUAL GLOSSARY	7
WORK SAFELY: MAKE A BACKUP	9
SOMETHING WENT WRONG – WHAT SHOULD I CHECK?	10
MY ERROR LOG	12
CHAPTER 1 · BUILDING THE PUBLIC WEBSITE	14
LESSON 1 · UNDERSTANDING THE PROFESSIONAL INVOICE GENERATOR	16
LESSON 2 · BUILDING THE LANDING PAGE	20
LESSON 3 · STYLING THE LANDING PAGE	34
LESSON 4 · ADDING THE RESPONSIVE NAVIGATION	45
LESSON 5 · COMPLETING THE PUBLIC WEBSITE	54
CHAPTER SUMMARY	63
CHAPTER MILESTONE	64
CHAPTER 2 · CONNECTING TO SUPABASE	66
LESSON 1 · CREATING YOUR SUPABASE PROJECT	68
LESSON 2 · UNDERSTANDING THE PROJECT URL AND PUBLISHABLE KEY	73
LESSON 3 · CREATING YOUR SUPABASE CONNECTION FILE	78
LESSON 4 · BUILDING A CONNECTION TEST PAGE	85
LESSON 5 · FINAL CONNECTION REVIEW	93
CHAPTER SUMMARY	98
CHAPTER MILESTONE	99
CHAPTER 3 · BUILDING THE COMPLETE AUTHENTICATION SYSTEM	100
LESSON 1 · UNDERSTANDING THE AUTHENTICATION FLOW	103
LESSON 2 · BUILDING THE COMPLETE AUTHENTICATION SYSTEM	107
LESSON 3 · AUDITING THE AUTHENTICATION SYSTEM	129

CHAPTER SUMMARY	134
CHAPTER MILESTONE	135
CHAPTER 4 · BUILDING THE INVOICE DATABASE	136
LESSON 1 · UNDERSTANDING THE INVOICE DATA MODEL	137
LESSON 2 · CREATING THE CUSTOMERS TABLE	141
LESSON 3 · CREATING THE INVOICES TABLE	145
LESSON 4 · CREATING THE INVOICE ITEMS TABLE	150
LESSON 5 · ADDING DATABASE CONSTRAINTS	154
LESSON 6 · ENABLING ROW LEVEL SECURITY	159
LESSON 7 · CREATING THE SELECT POLICIES	162
LESSON 8 · CREATING THE INSERT POLICIES	166
LESSON 9 · CREATING THE UPDATE POLICIES	170
LESSON 10 · CREATING THE DELETE POLICIES	175
LESSON 11 · TESTING THE INVOICE DATABASE	179
CHAPTER SUMMARY	184
CHAPTER MILESTONE	185
CHAPTER 5 · BUILDING THE INVOICE DASHBOARD	186
LESSON 1 · DESIGNING THE INVOICE WORKSPACE	188
LESSON 2 · SAVING AND SELECTING CUSTOMERS	194
LESSON 3 · BUILDING THE INVOICE FORM	201
LESSON 4 · CALCULATING AND SAVING INVOICES SECURELY	208
LESSON 5 · BUILDING INVOICE DASHBOARD STATISTICS	215
CHAPTER SUMMARY	220
CHAPTER MILESTONE	221
CHAPTER 6 · VIEWING INVOICES	222
LESSON 1 · BUILDING THE INVOICE HISTORY	224
LESSON 2 · BUILDING THE INVOICE DETAILS PAGE	230
LESSON 3 · TESTING THE INVOICE HISTORY	236
CHAPTER SUMMARY	240
CHAPTER MILESTONE	241
CHAPTER 7 · SEARCHING, FILTERING AND SORTING INVOICES	242

LESSON 1 · BUILDING INVOICE SEARCH, FILTERING AND SORTING	244
LESSON 2 · TESTING INVOICE SEARCH	250
CHAPTER SUMMARY	253
CHAPTER MILESTONE	254
CHAPTER 8 · EDITING INVOICES	255
LESSON 1 · BUILDING INVOICE EDITING	257
LESSON 2 · TESTING INVOICE EDITING	264
CHAPTER SUMMARY	268
CHAPTER MILESTONE	269
CHAPTER 9 · PRINTING AND DELETING INVOICES	270
LESSON 1 · BUILDING THE PRINTABLE INVOICE DOCUMENT	272
LESSON 2 · BUILDING AND TESTING SECURE INVOICE DELETION	278
CHAPTER SUMMARY	284
CHAPTER MILESTONE	285
CHAPTER 10 · FINAL APPLICATION REVIEW	286
LESSON 1 · COMPLETE SYSTEM TESTING	287
REFLECTION QUESTIONS	295
EXTENSION CHALLENGES	296
NEXT WORKBOOK	297

ABOUT THIS WORKBOOK

This workbook is a complete, self-contained project.

You do not need to own any of the other Prompt to Profit™ Software Workbooks to complete it successfully.

Every workbook in this series teaches you how to build one complete software application from start to finish. It contains all the explanations, prompts, instructions and practical exercises you need to complete that project independently.

Although some software development concepts naturally appear across multiple workbooks, every workbook starts from the beginning of its own project and assumes no prior knowledge of the other workbooks.

This means you can begin with this workbook even if you have not completed any other workbook.

If your immediate need is to build a Professional Invoice Generator, you can start here.

If you later decide to build additional software, each new workbook will teach you a different business application while reinforcing the software development skills you have already learned.

Together, the workbooks form a complete software development series. Individually, each workbook is a complete learning experience.

WELCOME

Welcome to Prompt to Profit™ Workbook 04.

In this workbook, you will build a complete Professional Invoice Generator from the ground up using HTML, CSS, Vanilla JavaScript, Supabase and Artificial Intelligence.

Throughout this project, you will learn how to build secure, database-driven business software by following the same step-by-step development process used in real-world software projects.

You will create a professional invoice application that allows a business to select a saved customer, add several invoice items, calculate totals, apply discounts and tax, save invoices, view previous invoices, edit them, delete them and print a professional invoice document.

Every feature is built one step at a time. You won't simply copy prompts into an AI tool. You'll learn why each feature is being built, how the different parts of the application work together, and how to communicate effectively with AI to build reliable software.

By the end of this workbook, you will have a fully functional Professional Invoice Generator that you can proudly include in your software development portfolio.

You will build the project using:

- HTML

- CSS
- Vanilla JavaScript
- Supabase
- ChatGPT
- Notepad
- A web browser

You do not need to install additional software.

You will continue working with complete files and clear Build Prompts.

Every new feature will preserve everything already built.

Complete the lessons in order.

Test each feature before moving forward.

Do not continue when an important feature is not working.

WHAT YOU WILL BUILD

By the end of this workbook, your Professional Invoice Generator will include:

PUBLIC WEBSITE

- Responsive navigation
- Hero section
- Business problem section
- Features section
- How It Works section
- Invoice preview
- Login and Register links

SUPABASE

- Supabase project
- Shared connection file
- Connection test page

AUTHENTICATION

- User registration
- Email verification
- User login
- Protected invoice pages
- Logout

INVOICE DATABASE

- Secure customers table
- Secure invoices table
- Secure invoice_items table
- User-owned invoices
- Row Level Security

INVOICE MANAGEMENT

- Save customer profiles
- Select saved customers
- Create invoices
- Add and remove invoice items
- Calculate subtotals automatically
- Apply percentage or fixed discounts
- Apply tax
- Save invoices
- View previous invoices
- Search, filter and sort invoices
- Edit invoices
- Delete invoices
- Print professional invoice documents

BUSINESS DASHBOARD

- Total invoices
- Draft invoices
- Sent invoices
- Paid invoices
- Loading, empty and error states

COMPLETION

- Complete application audit
- Two-account privacy testing
- Netlify deployment
- Live application testing
- Portfolio description
- Reflection questions
- Extension challenges

WORKBOOK STRUCTURE

This workbook is organised into ten chapters.

Chapter 1

Building the Public Website

Chapter 2

Connecting to Supabase

Chapter 3

Building the Complete Authentication System

Chapter 4

Building the Invoice Database

Chapter 5

Building the Invoice Dashboard

Chapter 6

Viewing Invoices

Chapter 7

Searching, Filtering and Sorting Invoices

Chapter 8

Editing Invoices

Chapter 9

Printing and Deleting Invoices

Chapter 10

Final Testing and Project Completion

Complete each chapter before moving to the next.

Every chapter depends on the work completed earlier.

LEARNER SUPPORT TOOLKIT

Keep this part of the workbook close while you build.

It explains common words, shows you how to protect your work, gives you a simple way to investigate problems, and provides a place to record errors and solutions.

You are not expected to remember everything immediately.

Return to these pages whenever you need them.

VISUAL GLOSSARY

HTML	The file that gives a web page its content and structure.
CSS	The file that controls colours, spacing, layout and appearance.
VANILLA JAVASCRIPT	JavaScript used without a framework. It controls what the application does.
BROWSER	The program used to open and test the application, such as Chrome or Edge.
FILE	One saved part of the project, such as index.html or dashboard.js.
FOLDER	The place where all files for one project are kept together.
FUNCTION	A named set of instructions that performs one task.
EVENT	An action the browser can notice, such as a click, form submission or key press.
VARIABLE	A named place used to hold information while the application is running.
URL	The address of a page, website or online service.
SUPABASE	The online service used for the database, authentication and security.
DATABASE	An organised place where the application stores information.
TABLE	A named part of the database that stores one type of information.
RECORD	One complete item saved inside a database table.
AUTHENTICATION	The process of creating accounts, signing in and identifying the current user.
SESSION	The saved sign-in state that allows the application to remember the current user.
ROW LEVEL SECURITY	Supabase rules that control which database records each user can access.

PUBLISHABLE KEY	The Supabase project key that browser-based applications are allowed to use.
DEPLOY	To place the complete project online so it has a live HTTPS website address.
DEBUGGING	The careful process of finding, understanding and correcting a problem.

WORK SAFELY: MAKE A BACKUP

A backup is a separate copy of your complete project folder.

Create a backup before replacing files that already contain working code.

1.

Close any project files that are open in Notepad.

2.

Find your complete project folder.

3.

Copy the folder.

4.

Paste the copy outside the working project folder.

Do not place the backup inside the folder you deploy.

5.

Rename the copy clearly.

For example:

Project Backup - Before Chapter 3 Lesson 2

6.

Open the backup folder.

Confirm that the expected files are inside it.

If a new change causes a serious problem, you can return to this working copy.

SOMETHING WENT WRONG — WHAT SHOULD I CHECK?

Problems are a normal part of building software.

Do not replace random pieces of code and do not continue to the next lesson.

Work through this checklist in order.

- ☐ Confirm that you opened the correct project folder.
- ☐ Confirm that every updated file was saved in Notepad.
- ☐ Check every filename carefully.
- ☐ Make sure an HTML, CSS or JavaScript file does not end with .txt.
- ☐ Confirm that you pasted the complete file returned by AI.
- ☐ Confirm that you did not paste an explanation, a code-block label or only part of a file.
- ☐ Check that stylesheet and JavaScript filenames match the names used inside the HTML.
- ☐ On Supabase pages, check that the scripts load in the order taught in the lesson.
- ☐ Read the exact message shown on the page.
- ☐ Open the browser Console and look for the first red error.

To open the Console:

Right-click the page.

Select:

Inspect

Then select:

Console

- ☐ Copy the complete first red error into your error log.
- ☐ Think about the last file you changed before the problem appeared.
- ☐ Compare that file with the lesson requirements.
- ☐ If necessary, restore the most recent working backup.

When asking AI for help, provide:

-
- What you were trying to do
 - What you expected to happen
 - What actually happened
 - The complete error message
 - The names of the files involved

Ask AI to return complete corrected files.

Do not ask for snippets.

MY ERROR LOG

Use this page whenever something does not work.

Writing down the problem helps you investigate it carefully and remember the solution.

ERROR 1

Date: _____

Chapter and lesson: _____

What I was trying to do:

What I expected to happen:

What actually happened:

First error message:

File or files involved:

What I checked or changed:

What fixed the problem:

ERROR 2

Date: _____

Chapter and lesson: _____

What I was trying to do:

What I expected to happen:

What actually happened:

First error message:

File or files involved:

What I checked or changed:

What fixed the problem:

CHAPTER 1

BUILDING THE PUBLIC WEBSITE

CHAPTER INTRODUCTION

Before a business owner creates an account, they need to understand what the software does and why it may be useful.

The public website is the first part of the application visitors will see.

It will explain the problems caused by poor invoice management and show how the software helps a business keep invoice information organised.

The landing page will also provide clear Login and Register links that will later connect visitors to the authentication system.

In this chapter, you will build the complete public website using three files:

index.html

styles.css

script.js

The HTML file will contain the structure.

The CSS file will control the appearance.

The JavaScript file will control the responsive navigation.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- The project folder
- index.html
- styles.css

- script.js
- Responsive navigation
- Three-line hamburger menu
- Hero section
- Business problem section
- Features section
- How It Works section
- Invoice dashboard preview
- Final call-to-action
- Footer
- Login link
- Register link
- Responsive mobile layout

By the end of the chapter, you will have a polished public website ready for the Supabase connection and authentication system.

LESSON 1

UNDERSTANDING THE PROFESSIONAL INVOICE GENERATOR

Estimated Time

10 minutes

WHAT YOU ARE BUILDING

You are building software that helps a business prepare professional invoices in one organised system.

A user will be able to create an account and:

- Select a saved customer
- Add several invoice items
- Enter a quantity and price for each item
- Calculate totals automatically
- Apply a discount
- Apply tax
- Save invoices
- Find previous invoices
- Edit invoices
- Delete invoices
- Print professional invoice documents

Every user's invoices will remain private.

WHY THIS MATTERS

Many small businesses prepare invoices using documents, spreadsheets or handwritten calculations.

This can create several problems.

An item may be forgotten.

A quantity may be multiplied incorrectly.

A discount or tax amount may be calculated incorrectly.

An old invoice may be difficult to find.

Two invoices may accidentally use the same number.

A Professional Invoice Generator solves these problems by keeping invoice details and items together, completing the calculations automatically, and saving every invoice in one secure database.

BEFORE YOU CONTINUE

Create a new folder on your computer.

Name it exactly:

Professional Invoice Generator

This folder will contain every file used throughout Workbook 04.

Do not copy or save files from any other workbook inside this folder.

Each software project should have its own folder.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Create the project folder and make sure you understand what the completed application will do.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson prepares you for the project.

SAVE YOUR FILES

No files are created during this lesson.

Confirm that the folder is named:

Professional Invoice Generator

TEST YOUR WORK

Open the new folder.

It should be empty.

Do not copy files from any other workbook into this folder.

You will build this project from the beginning.

CHECKPOINT

Before moving on, confirm that:

- ☒ You created the Professional Invoice Generator folder.
- ☒ The folder is empty.
- ☒ You understand the main purpose of the software.
- ☒ You understand that every user will have private invoices.

COMMON BEGINNER MISTAKES

One common mistake is placing files from different projects inside the same folder.

This can cause the browser to load the wrong stylesheet or JavaScript file.

Keep this project completely separate from any other workbook.

Another mistake is renaming the folder repeatedly during the project.

Choose the correct name now and keep it unchanged.

BEHIND THE SCENES

The application will eventually contain several connected systems.

The public website will explain the software.

Authentication will identify the user.

The invoices table will store records.

Row Level Security will protect those records.

JavaScript will connect the forms and dashboard to Supabase.

Every part will be added in the correct order.

THINK LIKE A SOFTWARE DESIGNER

Before building software, understand the real problem it is expected to solve.

This project is not simply a form that saves names and phone numbers.

It is a business system designed to make invoice information easier to store, find, update, and use.

That difference should guide every feature you build.

WHAT YOU LEARNED

In this lesson you learned:

- what a Professional Invoice Generator does
- why businesses need organised invoices
- what information the software will store
- why every software project should have its own folder
- why the application must protect each user's data

In the next lesson, you will build the first version of the public landing page.

CHAPTER 1

BUILDING THE PUBLIC WEBSITE

LESSON 2

BUILDING THE LANDING PAGE

Estimated Time

30 minutes

WHAT YOU ARE BUILDING

In this lesson, you will build the first version of the public website for your Professional Invoice Generator.

The landing page will explain what the software does, the business problems it solves, and the main features users will receive.

At this stage, the page will contain the complete HTML structure.

It will not look polished yet because the stylesheet has not been created.

The responsive navigation will also not work until the JavaScript file is added in a later lesson.

WHY THIS MATTERS

A business software application needs a clear public website.

Before someone creates an account, they should be able to understand:

- What the software does
- Who it is designed for
- What problems it solves
- What features are available
- What action they should take next

A well-structured landing page helps visitors understand the value of the application before they register.

BEFORE YOU CONTINUE

Confirm that your project folder is named:

Professional Invoice Generator

Open the folder.

It should still be empty.

You are about to create the first file:

index.html

When saving the file in Notepad:

- Click File.
- Click Save As.
- Open the Professional Invoice Generator folder.
- Change Save as type to All Files.
- Enter the filename exactly as:

index.html

Make sure the file is not saved as:

index.html.txt

BUILD PROMPT

PROMPT STARTS HERE

Create one complete HTML5 file named:

index.html

The file is the public landing page for a Professional Invoice Generator designed for small businesses.

Return the complete index.html file only.

Do not return explanations.

Do not return snippets.

Do not return CSS.

Do not return JavaScript.

Do not omit any section.

FILE CONNECTIONS

Inside the head section, include this stylesheet link exactly:

```
<link rel="stylesheet" href="styles.css">
```

Immediately before the closing body tag, include this JavaScript link exactly:

```
<script src="script.js"></script>
```

The files `styles.css` and `script.js` will be created in later lessons.

Do not include CSS inside `index.html`.

Do not include JavaScript inside `index.html`.

GENERAL HTML REQUIREMENTS

- Use proper HTML5 structure.
- Include the viewport meta tag.
- Include an appropriate page title.
- Use semantic HTML elements.
- Use clear and meaningful class names.
- Use section IDs that match the navigation links.
- Keep the content professional and easy to understand.
- Do not use external images.
- Do not use external icons.
- Do not use any framework.
- Do not use placeholder text such as Lorem Ipsum.

PAGE STRUCTURE

Build the sections below in this exact order.

1. HEADER AND NAVIGATION

Create a professional header.

Include a clickable logo that displays:

Invoice Records

The logo must link to:

index.html

Include these navigation links:

Home

Features

How It Works

Login

Register

The links should point to:

Home:

#home

Features:

#features

How It Works:

#how-it-works

Login:

login.html

Register:

register.html

Style-related classes should already be added so CSS can later make Login a secondary action and Register the main action.

Add a mobile hamburger button.

The hamburger button must:

- Use a button element.
- Include exactly three empty span elements.
- Include an aria-label that explains its purpose.
- Include:

aria-expanded="false"

- Include:

aria-controls

pointing to the navigation menu.

Do not build the hamburger lines using CSS pseudo-elements.

The three span elements must exist inside the HTML.

2. HERO SECTION

Give the section the ID:

home

Include:

- A clear headline explaining that the software helps businesses organise invoice information.
- A short supporting paragraph.
- A primary button labelled:

Get Started

The button must link to:

register.html

- A secondary button labelled:

See How It Works

The button must link to:

#how-it-works

Also include an invoice management preview built entirely with HTML.

The preview should contain simple cards or panels representing:

- Total Invoices
- Draft Invoices
- Paid Invoices
- A short invoice list

Do not use an image.

3. BUSINESS PROBLEM SECTION

Create a section explaining the problems businesses face when invoice information is scattered across:

- Notebooks
- Phone contacts
- Spreadsheets
- Messaging applications

- Employees' personal devices

Explain that scattered records can lead to:

- Lost invoice details
- Repeated invoice numbers
- Incorrect totals
- Slow invoice preparation
- Difficulty finding overdue invoices

Keep the wording clear and concise.

4. FEATURES SECTION

Give the section the ID:

features

Include six feature cards.

The cards should explain:

1. Saved Customers
2. Multiple Invoice Items
3. Automatic Totals
4. Discounts and Tax
5. Professional Printing
6. Search, Filters and Sorting

Each card should include:

- A short title
- A brief explanation

Do not use external icons.

You may use simple text labels or small decorative HTML elements.

5. HOW IT WORKS SECTION

Give the section the ID:

how-it-works

Explain the process in four clear steps:

1. Create an Account

2. Save and Select a Customer

3. Prepare and Save an Invoice

4. Find, Edit and Print Invoices

Each step should have:

- A number
- A heading
- A short explanation

6. DASHBOARD PREVIEW SECTION

Create a larger dashboard preview using HTML only.

Include:

- Four summary cards
- An invoice search area
- Filter controls
- Three sample invoice cards

Each sample invoice card should display example information such as:

- Invoice number
- Customer name
- Status
- Issue date
- Due date
- Final total
- View Invoice button

The sample data must be clearly fictional.

Do not use real personal information.

The buttons are only part of the visual preview and do not need to work.

7. FINAL CALL-TO-ACTION SECTION

Encourage the visitor to organise invoices in one secure system.

Include a button labelled:

Create Your Account

The button must link to:

register.html

8. FOOTER

Include:

- Logo or software name
- A short description
- Quick links
- Login link pointing to login.html
- Register link pointing to register.html
- Copyright text

Use a JavaScript-friendly span with an ID for the copyright year if appropriate, but do not include JavaScript inside the page.

ACCESSIBILITY REQUIREMENTS

- Use heading levels in a logical order.
- Add descriptive text to links and buttons.
- Use aria-labels where necessary.
- Make the hamburger button accessible.
- Use meaningful link text.
- Avoid empty links.

IMPORTANT

The Login links must point to:

login.html

The Register and Get Started links must point to:

register.html

The stylesheet link must be:

```
<link rel="stylesheet" href="styles.css">
```

The JavaScript link must be:

```
<script src="script.js"></script>
```


Return one complete index.html file from beginning to end.

Do not include unfinished comments.

Do not return any other file.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return one complete file:

index.html

The file should contain:

- ☒ Correct HTML5 structure
- ☒ Stylesheet link to styles.css
- ☒ JavaScript link to script.js
- ☒ Header and navigation
- ☒ Three-span hamburger button
- ☒ Hero section
- ☒ Business problem section
- ☒ Six feature cards
- ☒ Four-step How It Works section
- ☒ Dashboard preview
- ☒ Final call-to-action
- ☒ Footer
- ☒ Login links pointing to login.html
- ☒ Register links pointing to register.html

The response should begin with:

<!DOCTYPE html>

It should not begin with an explanation.

SAVE YOUR FILES

Copy the complete HTML code returned by ChatGPT.

Open Notepad.

Paste the complete code into the empty document.

Click File.

Click Save As.

Open your Professional Invoice Generator folder.

Choose:

Save as type: All Files

Enter the filename exactly as:

index.html

Click Save.

Close Notepad.

Open the project folder and confirm that the file appears as:

index.html

Make sure it is not named:

index.html.txt

TEST YOUR WORK

Open your Professional Invoice Generator folder.

Double-click:

index.html

The page should open in your browser.

It will look plain because styles.css does not exist yet.

That is expected.

Scroll through the complete page.

Confirm that you can see:

- ☒ Header
- ☒ Navigation
- ☒ Hero section
- ☒ Business problem section
- ☒ Features section
- ☒ How It Works section
- ☒ Dashboard preview
- ☒ Final call-to-action
- ☒ Footer

Click:

Features

The browser should move to the Features section.

Click:

How It Works

The browser should move to the How It Works section.

Click:

Login

The browser may show that login.html does not exist.

This is expected.

Click:

Register

The browser may show that register.html does not exist.

This is also expected.

Those pages will be created later as part of the complete authentication system.

CHECKPOINT

Before moving on, confirm that:

- ✓ index.html exists.
- ✓ The browser opens the file as a webpage.
- ✓ The page does not display HTML code as plain text.
- ✓ Every required section is present.
- ✓ The hamburger button contains three span elements.
- ✓ Login points to login.html.
- ✓ Register points to register.html.
- ✓ styles.css is linked.
- ✓ script.js is linked.
- ✓ The filename is not index.html.txt.

COMMON BEGINNER MISTAKES

A common mistake is saving the file using the wrong extension.

If the file is saved as:

index.html.txt

the browser may not treat it as a webpage.

Open the project folder and check the complete filename carefully.

Another mistake is removing the links to styles.css and script.js because those files do not exist yet.

Do not remove them.

They are being linked now so the files will connect correctly when they are created.

Some students also change:

login.html

to:

login

or change:

register.html

to:

signup.html

Do not do this.

The filenames must remain consistent because later pages and redirects will depend on them.

BEHIND THE SCENES

The landing page already refers to files that will be created in later lessons.

This is intentional.

The browser reads:

```
<link rel="stylesheet" href="styles.css">
```

and looks for a file named styles.css inside the same folder.

It also reads:

```
<script src="script.js"></script>
```

and looks for script.js inside the same folder.

The page remains usable even though those files are not present yet, but it will not have its final appearance or interactive navigation until they are created.

THINK LIKE A SOFTWARE DESIGNER

A public website should help visitors understand the software before asking them to create an account.

The sections should answer the visitor's questions in a sensible order.

First, explain the software.

Next, explain the business problem.

Then show the features and how the system works.

Finally, ask the visitor to register.

This creates a clear journey instead of presenting disconnected information.

CODE-READING QUESTION

Open index.html. Find the part of the page added for building the landing page. Which HTML element starts that part?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- create the main HTML file for a business application
- structure a professional landing page
- connect future CSS and JavaScript files correctly
- prepare navigation for future authentication pages
- build a hamburger button with three span elements
- save an HTML file correctly using Notepad

In the next lesson, you will create styles.css and transform the plain landing page into a polished, responsive business website.

CHAPTER 1**BUILDING THE PUBLIC WEBSITE****LESSON 3**

STYLING THE LANDING PAGE

Estimated Time

35 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create the stylesheet for your Professional Invoice Generator.

The HTML structure is already complete.

You will now transform it into a professional business website suitable for a real software product.

The responsive navigation will be completed in the next lesson when you create the JavaScript file.

WHY THIS MATTERS

Without CSS, a webpage contains only structure.

CSS controls:

- Colours
- Typography
- Layout
- Spacing
- Buttons
- Cards
- Responsive behaviour

A professional design helps visitors understand the software more quickly and creates confidence in the product.

BEFORE YOU CONTINUE

Before starting this lesson, confirm that:

- ✓ index.html opens correctly.
- ✓ Every landing page section appears.
- ✓ The browser does not display HTML code as plain text.

Open index.html in Notepad.

Copy the complete code.

You will paste it into the Build Prompt so ChatGPT can style the exact HTML structure you have already created.

BUILD PROMPT

PROMPT STARTS HERE

I already have a complete HTML file named:

index.html

for my Professional Invoice Generator.

Create one complete CSS file named:

styles.css

Return only the complete styles.css file.

Do not return HTML.

Do not return JavaScript.

Do not return explanations.

Do not return snippets.

Everything already built must continue working.

The stylesheet must match the exact HTML structure that I will paste below.

The HTML already contains this stylesheet link:

```
<link rel="stylesheet" href="styles.css">
```

GENERAL DESIGN

Create a clean, modern business software design.

Use a professional colour palette suitable for business applications.

Create CSS variables inside:

:root

Include variables for:

- Primary colour
- Secondary colour
- Accent colour
- Background colour
- Card colour
- Text colour
- Border colour
- Shadow
- Border radius
- Content width

Use:

box-sizing: border-box;

for every element.

Remove unnecessary browser margins and padding.

Use a professional system font stack.

Do not import fonts.

GENERAL LAYOUT

Create a centred content container.

Give each major section comfortable spacing.

Prevent content from touching the edges of the screen.

Create balanced spacing between headings, paragraphs, cards and buttons.

HEADER

Create a professional header.

Display:

Invoice Records

clearly as the brand.

Arrange navigation horizontally on larger screens.

Style:

Login

as a secondary button.

Style:

Register

as the primary action.

Include hover and keyboard focus styles.

HAMBURGER BUTTON

Style the existing hamburger button.

The button already contains exactly three span elements.

Do not create additional lines using CSS pseudo-elements.

Hide the hamburger button on larger screens.

Display it on smaller screens.

Prepare the navigation menu so it becomes visible whenever JavaScript adds an:

active

class.

Do not write JavaScript.

HERO SECTION

Create an attractive hero layout.

Display the content and dashboard preview side by side on wider screens.

Stack them on smaller screens.

Make:

Get Started

the strongest visual button.

Style:

See How It Works

as a secondary button.

BUSINESS PROBLEM SECTION

Give this section a subtle visual difference from the hero section.

Use generous spacing.

Make the problem easy to read.

FEATURES SECTION

Display the six feature cards in a responsive grid.

Each card should include:

- Rounded corners
- Professional shadow
- Comfortable spacing
- Subtle hover effect

HOW IT WORKS SECTION

Display the four steps using attractive cards.

Clearly distinguish the step number from the description.

DASHBOARD PREVIEW

Style the HTML dashboard preview so it resembles a real business dashboard.

Style:

- Summary cards
- Search area
- Invoice cards
- View Invoice buttons

Do not depend on images.

FINAL CALL-TO-ACTION

Create a strong closing section.

Highlight the:

Create Your Account

button.

FOOTER

Create a professional footer.

Arrange footer content neatly.

Style footer navigation links.

RESPONSIVE DESIGN

Create responsive layouts for:

Desktop

Tablet

Mobile

On smaller screens:

Hide the desktop navigation.

Display the hamburger button.

Prepare the navigation to become visible when JavaScript adds the:

active

class.

Stack multi-column layouts vertically.

Prevent horizontal scrolling.

Keep buttons large enough to tap comfortably.

ACCESSIBILITY

Create visible keyboard focus styles.

Maintain good colour contrast.

Respect reduced-motion preferences where appropriate.

TECHNICAL REQUIREMENTS

Match the exact class names used in the supplied HTML.

Do not invent a different HTML structure.

Do not use CSS frameworks.

Do not use external fonts.

Return one complete:

styles.css

file.

Here is my complete index.html file:

[PASTE YOUR COMPLETE INDEX.HTML HERE]

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return one complete file:

styles.css

The file should:

- ☒ Match the HTML.
- ☒ Style every section.
- ☒ Support desktop, tablet and mobile layouts.
- ☒ Prepare the navigation for JavaScript.
- ☒ Include no HTML.
- ☒ Include no JavaScript.

SAVE YOUR FILES

Copy the complete CSS returned by ChatGPT.

Open Notepad.

Paste the code into a new document.

Click File.

Click Save As.

Open your Professional Invoice Generator folder.

Choose:

Save as type:

All Files

Enter the filename exactly as:

styles.css

Click Save.

Close Notepad.

Your project folder should now contain:

index.html

styles.css

TEST YOUR WORK

Open:

index.html

If it is already open in your browser, refresh the page.

The landing page should now have a professional appearance.

Confirm that:

- ☒ Header is styled.
- ☒ Navigation is styled.
- ☒ Hero section looks professional.
- ☒ Dashboard preview resembles business software.
- ☒ Feature cards display correctly.
- ☒ Buttons look professional.
- ☒ Footer is styled.

Now reduce the browser width.

Confirm that:

- ☒ Layout adjusts correctly.

☒ Hamburger button appears.

☒ Desktop navigation disappears.

The hamburger button does not need to work yet.

That functionality will be added in the next lesson.

CHECKPOINT

Before moving on, confirm that:

☒ styles.css exists.

☒ The landing page is styled.

☒ Responsive layouts work.

☒ Hamburger button appears on smaller screens.

☒ Buttons display correctly.

☒ Dashboard preview looks professional.

☒ Footer is styled.

COMMON BEGINNER MISTAKES

A common mistake is saving the file as:

styles.css.txt

Always choose:

Save as type:

All Files

Another common mistake is changing class names inside the CSS.

The stylesheet must match the HTML structure exactly.

If ChatGPT returns styles that do not match your HTML, provide your complete index.html again and ask for a complete updated styles.css file.

Do not ask for snippets.

BEHIND THE SCENES

When your browser opens index.html, it immediately reads:

```
<link rel="stylesheet" href="styles.css">
```

It then loads every CSS rule inside styles.css and applies those rules to matching HTML elements.

This is why HTML and CSS must always stay in sync.

If an HTML class changes but the CSS is not updated, the styling may stop working.

THINK LIKE A SOFTWARE DESIGNER

Business software should communicate trust.

Professional colours, balanced spacing, consistent buttons, and organised layouts help users feel confident before they even create an account.

Good design is not decoration.

Good design makes software easier to understand and easier to use.

CODE-READING QUESTION

Open styles.css. Find one style rule added for styling the landing page. Which selector does the rule use?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a professional stylesheet
- style a business landing page
- prepare responsive layouts

- style dashboard preview cards
- prepare the navigation for JavaScript
- keep HTML and CSS working together

In the next lesson, you will create `script.js` and make the responsive navigation fully functional.

CHAPTER 1

BUILDING THE PUBLIC WEBSITE

LESSON 4

ADDING THE RESPONSIVE NAVIGATION

Estimated Time

30 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create the JavaScript that makes the responsive navigation work.

Your landing page already contains:

- HTML structure
- Professional styling
- A three-line hamburger button

The only missing piece is the behaviour.

When the browser window becomes narrow, visitors should be able to open and close the navigation menu using the hamburger button.

WHY THIS MATTERS

Many people will visit your website using a mobile phone or tablet.

A navigation menu that works well on a large computer screen may become difficult to use on a smaller device.

Responsive navigation solves this problem by allowing users to open and close the menu whenever they need it.

This creates a cleaner layout and improves the overall user experience.

BEFORE YOU CONTINUE

Before starting this lesson, confirm that your project folder contains:

index.html

styles.css

Open index.html in your browser.

Reduce the browser width.

You should see:

☒ The hamburger button.

☒ The normal navigation hidden.

The hamburger button will not do anything yet.

That is expected.

Open index.html in Notepad.

Copy the complete code.

Also open styles.css in Notepad and copy the complete code.

You will paste both files into the Build Prompt so ChatGPT can use the exact IDs and class names already in your project.

BUILD PROMPT

PROMPT STARTS HERE

I already have these files in my project:

index.html

styles.css

Everything already built must continue working.

Create one complete JavaScript file named:

script.js

Return only the complete script.js file.

Do not return explanations.

Do not return snippets.

Do not return HTML.

Do not return CSS.

GENERAL REQUIREMENTS

- Use plain Vanilla JavaScript.
- Do not use any framework.
- Do not use external libraries.
- Do not use Supabase.
- Do not add authentication code.
- Do not add invoice management code.

RESPONSIVE NAVIGATION

- Use the exact HTML structure that I provide.
- Find:
 - The hamburger button.
 - The navigation menu.
- Use the exact IDs or class names already present.
- When the hamburger button is clicked:
 - Open the navigation menu.
 - Add the existing active class expected by styles.css.
- When the hamburger button is clicked again:
 - Close the navigation menu.
 - Remove the active class.

ACCESSIBILITY

- Update:
 - aria-expanded
 - correctly.
- When the menu opens:
 - aria-expanded="true"
- When the menu closes:
 - aria-expanded="false"
- Do not remove any accessibility attributes already present.

CLOSING THE MENU

When the visitor clicks any navigation link inside the mobile menu:

- Close the menu.

When the visitor presses the Escape key:

- Close the menu.

When the visitor clicks anywhere outside the navigation:

- Close the menu.

When the browser window becomes wider than the mobile layout:

- Close the mobile menu automatically.

SMOOTH NAVIGATION

Allow links pointing to sections inside:

`index.html`

to move smoothly.

Do not interfere with links pointing to:

`login.html`

`register.html`

Those links must continue opening normally.

CODE QUALITY

Wait until the page has loaded before selecting HTML elements.

Check that required elements exist before attaching event listeners.

Use clear variable names.

Avoid duplicate event listeners.

Do not include unfinished comments.

Return one complete `script.js` file.

IMPORTANT

Everything already built must continue working.

Use the exact IDs and class names already present inside my HTML.

Do not invent different selectors.

The active class must match the class already prepared inside styles.css.

Here is my complete index.html file:

[PASTE YOUR COMPLETE INDEX.HTML HERE]

Here is my complete styles.css file:

[PASTE YOUR COMPLETE STYLES.CSS HERE]

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return one complete file:

script.js

The file should:

- ☒ Open the mobile navigation.
- ☒ Close the mobile navigation.
- ☒ Update aria-expanded.
- ☒ Close after selecting a navigation link.
- ☒ Close after pressing Escape.
- ☒ Close after clicking outside the navigation.
- ☒ Close automatically when returning to desktop width.
- ☒ Leave Login and Register links working normally.

SAVE YOUR FILES

Copy the complete JavaScript returned by ChatGPT.

Open Notepad.

Paste the code into a new document.

Click File.

Click Save As.

Open your Professional Invoice Generator folder.

Choose:

Save as type:

All Files

Enter the filename exactly as:

script.js

Click Save.

Close Notepad.

Your project folder should now contain:

index.html

styles.css

script.js

Make sure the file is not named:

script.js.txt

TEST YOUR WORK

Open:

index.html

If it is already open, refresh the browser.

Reduce the browser width until the hamburger button appears.

Click the hamburger button.

The navigation menu should open.

Click the hamburger button again.

The navigation menu should close.

Open the menu.

Click:

Features

The menu should close automatically.

The browser should move to the Features section.

Open the menu again.

Press the Escape key.

The menu should close.

Open the menu again.

Click outside the navigation.

The menu should close.

Finally, widen the browser window.

The mobile menu should close automatically and the normal desktop navigation should appear again.

Click:

Login

The browser may display a page-not-found message.

This is expected because login.html has not yet been created.

Click:

Register

The browser may also display a page-not-found message.

This is expected because register.html will be created later in Workbook 04.

CHECKPOINT

Before moving on, confirm that:

- ☒ script.js exists.
- ☒ The hamburger button opens the menu.
- ☒ The hamburger button closes the menu.
- ☒ Navigation links close the menu.
- ☒ Escape closes the menu.

- ✓ Clicking outside closes the menu.
- ✓ Returning to desktop width closes the mobile menu.
- ✓ Login still points to login.html.
- ✓ Register still points to register.html.

COMMON BEGINNER MISTAKES

A common mistake is changing the class name used by JavaScript without updating the CSS.

The JavaScript and CSS must both use the same active class.

Another common mistake is searching for HTML elements using IDs or class names that do not exist.

Always use the exact HTML already present in your project.

If the menu does not open, provide ChatGPT with both your complete index.html and styles.css files and ask it to regenerate the complete script.js file.

Do not ask for only a small correction.

BEHIND THE SCENES

JavaScript does not create a second navigation menu.

Instead, it changes the state of the menu that already exists inside index.html.

When the active class is added, CSS displays the menu.

When the active class is removed, CSS hides it again.

The browser simply switches between those two states.

THINK LIKE A SOFTWARE DESIGNER

Responsive navigation should feel natural.

Users should never wonder how to close the menu or whether the application has stopped responding.

Good software provides predictable behaviour in every situation.

Small details like automatically closing the menu after a navigation link is selected make an application feel much more polished.

CODE-READING QUESTION

Open script.js. Find one function that supports adding the responsive navigation. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a JavaScript file
- control responsive navigation
- update accessibility attributes
- respond to user interactions
- coordinate HTML, CSS and JavaScript
- improve the mobile experience

In the next lesson, you will review the complete public website, improve its overall quality, and prepare it for the Supabase connection in Chapter 2.

CHAPTER 1

BUILDING THE PUBLIC WEBSITE

LESSON 5

COMPLETING THE PUBLIC WEBSITE

Estimated Time

30 minutes

WHAT YOU ARE BUILDING

In this lesson, you will review and improve the complete public website for your Professional Invoice Generator.

You have already built:

- `index.html`
- `styles.css`
- `script.js`

Your website is functional, but like most software, it can still be improved.

Rather than adding new features, you will refine what already exists while preserving every working part of the project.

WHY THIS MATTERS

Professional software is rarely perfect after the first version.

Developers continuously improve layouts, wording, accessibility, responsiveness, spacing, and usability.

These improvements make the software easier to understand and more pleasant to use without changing its purpose.

Learning how to improve existing software safely is one of the most valuable software development skills.

BEFORE YOU CONTINUE

Confirm that your Professional Invoice Generator folder contains:

index.html

styles.css

script.js

Open index.html in your browser.

Confirm that:

☒ The landing page opens.

☒ The page is fully styled.

☒ The responsive navigation works.

☒ The Login link points to:

login.html

☒ The Register link points to:

register.html

Do not continue until everything above works correctly.

BUILD PROMPT

PROMPT STARTS HERE

I already have a complete public website for my Professional Invoice Generator.

Everything already built must continue working.

Do not remove any existing functionality.

Return complete updated versions of every affected file.

Do not return snippets.

Do not return explanations.

Return complete updated versions of:

index.html

styles.css

script.js

only if improvements are required.

GENERAL GOAL

Review the complete public website and improve its overall quality while preserving every existing feature.

INDEX.HTML

Keep every existing section.

Do not remove:

- Header
- Navigation
- Hero
- Business Problem
- Features
- How It Works
- Dashboard Preview
- Final Call-to-Action
- Footer

Improve where appropriate:

- Heading wording
- Paragraph wording
- Accessibility labels
- Semantic HTML

Do not rename IDs unless absolutely necessary.

Keep:

```
<link rel="stylesheet" href="styles.css">
```

Keep:

```
<script src="script.js"></script>
```

Continue pointing:

Login

to:

login.html

Continue pointing:

Register

to:

register.html

STYLES.CSS

Keep every existing style.

Improve where appropriate:

- Consistent spacing
- Typography
- Button styling
- Card alignment
- Section spacing
- Mobile layout
- Dashboard preview appearance
- Footer layout

Improve keyboard focus styles where appropriate.

Maintain good colour contrast.

Do not redesign the page completely.

Refine the existing design.

SCRIPT.JS

Keep every existing feature working.

Review the JavaScript for:

- Duplicate event listeners
- Repeated logic
- Unnecessary code
- Accessibility improvements

Continue supporting:

- Responsive navigation
- Hamburger menu
- Navigation closing
- Escape key

- Outside click
 - Desktop resize handling
- Do not add authentication code.
- Do not add Supabase code.
- Do not add invoice management code.

QUALITY REVIEW

Review the complete website and ensure:

- Navigation feels smooth.
- Buttons are consistent.
- Messages and wording are professional.
- Layout spacing is balanced.
- Mobile experience is clean.
- Keyboard navigation still works.

IMPORTANT

- Everything already built must continue working.
- Do not remove features.
- Do not rename files.
- Return complete updated files only.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

- ChatGPT should return complete updated versions of any files that require improvement.
- Every returned file should be complete from beginning to end.
- Nothing that already works should stop working.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

If ChatGPT returns an updated version of any file:

Open that file in Notepad.

Delete the existing contents.

Paste the complete updated code.

Click File.

Click Save.

Repeat this process for every updated file.

Do not create duplicate files.

Always replace the complete contents of the existing file.

Your project folder should still contain only:

index.html

styles.css

script.js

TEST YOUR WORK

Refresh:

index.html

Review the complete landing page.

Confirm that:

- ☒ The header still appears correctly.
- ☒ Navigation still works.
- ☒ The hamburger menu still works.
- ☒ Every section remains present.
- ☒ Buttons remain consistent.
- ☒ The dashboard preview still looks professional.
- ☒ The footer still appears correctly.

Now reduce the browser width.

Confirm that:

- ☒ The layout adjusts correctly.
- ☒ The hamburger menu still works.
- ☒ No content runs outside the page.
- ☒ No unnecessary horizontal scrolling appears.

Finally, click:

Home

Features

How It Works

Login

Register

Confirm that:

- ☒ Internal navigation still works.
- ☒ Login still points to login.html.
- ☒ Register still points to register.html.

CHECKPOINT

Before moving on, confirm that:

- ☒ The landing page looks professional.

- ✓ Every section remains present.
- ✓ Navigation still works.
- ✓ Responsive behaviour still works.
- ✓ Login and Register links remain correct.
- ✓ No existing functionality has been lost.

COMMON BEGINNER MISTAKES

Some beginners ask ChatGPT to improve only a small section of a file.

That often causes important code to disappear.

Always request the complete updated file.

Another common mistake is accepting improvements without testing the complete website afterwards.

Even small changes can accidentally affect responsive layouts or navigation.

Refresh the browser and test the whole page after every update.

BEHIND THE SCENES

As software projects become larger, developers rarely rebuild them from scratch.

Instead, they improve existing files while preserving the features that already work.

Throughout this workbook, you will continue following this same approach.

Each Build Prompt will protect everything already built while adding or refining only what is necessary.

THINK LIKE A SOFTWARE DESIGNER

Professional software evolves through continuous improvement.

Instead of asking,

"What can I build next?"

experienced developers often ask,

"What can I make better?"

Learning to improve existing software without breaking it is one of the habits that separates experienced developers from beginners.

CODE-READING QUESTION

Open index.html. Find the part of the page added for completing the public website. Which HTML element starts that part?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- review an existing project
- improve a public website safely
- preserve existing functionality
- refine layouts and wording
- test a website after improvements

CHAPTER SUMMARY

Congratulations.

You have completed the public website for your Professional Invoice Generator.

Your application now has a professional landing page that explains the software clearly and guides visitors towards creating an account.

You also built a responsive navigation system that works across different screen sizes.

Most importantly, you have followed a professional step-by-step workflow.

You built the application one reliable step at a time, testing each stage before moving forward.

CHAPTER MILESTONE

By the end of Chapter 1, you have successfully built:

- ✓ Professional landing page
- ✓ Responsive navigation
- ✓ Three-line hamburger menu
- ✓ Hero section
- ✓ Business problem section
- ✓ Features section
- ✓ How It Works section
- ✓ Dashboard preview
- ✓ Final call-to-action
- ✓ Footer
- ✓ Login link
- ✓ Register link
- ✓ Responsive layout
- ✓ HTML, CSS and JavaScript working together

Your project folder should now contain:

index.html

styles.css

script.js

TRANSITION TO CHAPTER 2

Your public website is now complete.

Visitors can learn about your software and navigate through the landing page.

The next step is to connect your application to Supabase.

In Chapter 2, you will create your Supabase project, build a shared connection file, test the connection, and prepare the application for the complete authentication system that follows.

Once that foundation is complete, you will begin building secure user accounts and protected invoice management features.

CHAPTER 2

CONNECTING TO SUPABASE

CHAPTER INTRODUCTION

Your public website is now complete.

Visitors can learn about your Professional Invoice Generator, understand how it works, and choose to register for an account.

The next step is to give your application a secure online backend.

That backend is Supabase.

Supabase will be responsible for:

- User authentication
- Invoice data storage
- Database security
- Protecting every user's invoices

During this chapter, you will connect your website to your own Supabase project.

Although visitors will not register until Chapter 3, everything required to support registration and invoice management will already be in place.

Think of this chapter as laying the foundation before constructing the building.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will:

- Create a Supabase project
- Locate the Project URL
- Locate the Publishable Key

- Create a shared connection file
- Build a connection test page
- Verify that your application communicates successfully with Supabase

By the end of the chapter, your project will be fully prepared for authentication and invoice management.

LESSON 1

CREATING YOUR SUPABASE PROJECT

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create a new Supabase project specifically for your Professional Invoice Generator.

Each workbook should have its own Supabase project.

Keeping projects separate makes them easier to manage and prevents one application from accidentally affecting another.

WHY THIS MATTERS

Your Professional Invoice Generator needs a secure online backend.

Without one, users cannot:

- Create accounts
- Sign in
- Store invoices
- Retrieve invoices later

Supabase provides all of these services.

BEFORE YOU CONTINUE

Make sure you have:

- ☒ A working internet connection.
- ☒ A web browser.
- ☒ Your Professional Invoice Generator folder.

No files will be created during this lesson.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, create your Supabase project.

Follow these steps.

1.

Open your web browser.

2.

Visit:

<https://supabase.com>

3.

Create a free account if you do not already have one.

4.

Sign in.

5.

Click:

New Project

6.

Choose your organisation.

7.

Enter a project name.

For example:

Professional Invoice Generator

8.

Create a strong database password.

Store it somewhere safe.

Do not share it with anyone.

Your application will not use this password directly.

9.

Choose the region closest to you.

10.

Click:

Create New Project

Wait while Supabase prepares the project.

This may take several minutes.

Do not continue until the project is ready.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson is completed inside your web browser.

SAVE YOUR FILES

No project files are created during this lesson.

TEST YOUR WORK

After the project has been created successfully:

Open the project dashboard.

Confirm that:

- ☒ The project opens.
- ☒ The project status shows that it is ready.

CHECKPOINT

Before moving on, confirm that:

- ☒ Your Supabase account exists.
- ☒ Your new project has been created.
- ☒ The project dashboard opens successfully.
- ☒ The project is ready.

COMMON BEGINNER MISTAKES

Some students close the browser before Supabase finishes creating the project.

Always wait until the project is fully ready.

Another common mistake is forgetting the database password.

Store it somewhere safe before leaving this page.

Remember that your website will not use this password.

Later lessons will use the Publishable Key instead.

BEHIND THE SCENES

When Supabase creates your project, it automatically prepares:

- A PostgreSQL database
- User authentication
- Storage services
- Security settings
- APIs

These services work together to support the software you are building.

THINK LIKE A SOFTWARE DESIGNER

Professional developers normally create a separate backend for each application.

Keeping projects separate reduces confusion, improves security, and makes future maintenance much easier.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a Supabase project
- prepare an online backend

- understand why every software project should have its own database

In the next lesson, you will locate the Project URL and Publishable Key that allow your Professional Invoice Generator to communicate securely with Supabase.

CHAPTER 2**CONNECTING TO SUPABASE****LESSON 2**

UNDERSTANDING THE PROJECT URL AND PUBLISHABLE KEY

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will locate the two pieces of information your Professional Invoice Generator needs to communicate with Supabase.

These are:

- The Project URL
- The Publishable Key

In the next lesson, you will place these values inside a reusable connection file that every page in your application will use.

WHY THIS MATTERS

Your website needs to know where your Supabase project is located before it can communicate with it.

The Project URL identifies your Supabase project.

The Publishable Key allows your website to communicate with that project securely.

These two values will be used throughout this workbook whenever your application:

- Registers a user
- Signs in a user
- Loads invoices
- Saves invoices
- Updates invoices

- Deletes invoices

BEFORE YOU CONTINUE

Confirm that:

- ☒ Your Supabase project has finished creating.
- ☒ You are signed in to Supabase.
- ☒ Your project dashboard is open.

No files will be created during this lesson.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, locate your Project URL and Publishable Key.

Follow these steps carefully.

1.

Open your Supabase project.

2.

From the left-hand menu, click:

Project Settings

3.

Click:

Data API

Locate:

Project URL

It will look similar to:

<https://your-project-id.supabase.co>

Copy the complete value.

Do not change it.

Next, locate:

Publishable Key

Copy the complete key.

Store both values somewhere safe.

You will use them in the next lesson.

IMPORTANT

Your Project URL must end with:

.supabase.co

Do not add:

/rest/v1/

The correct format is:

<https://your-project-id.supabase.co>

IMPORTANT

Use only the:

Publishable Key

Never use:

- Service Role Key
- Secret Key
- Database password

Only the Publishable Key belongs inside your project files.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson is completed inside your Supabase dashboard.

SAVE YOUR FILES

No project files are created during this lesson.

Keep your Project URL and Publishable Key ready for the next lesson.

TEST YOUR WORK

Confirm that you have copied:

- ☒ Your Project URL
- ☒ Your Publishable Key

Check the Project URL carefully.

It should end with:

.supabase.co

There should be nothing after that.

CHECKPOINT

Before moving on, confirm that:

- ☒ You have located your Project URL.
- ☒ You have located your Publishable Key.
- ☒ You copied the Publishable Key.
- ☒ You did not copy a secret key.
- ☒ Your Project URL does not contain:

/rest/v1/

COMMON BEGINNER MISTAKES

One common mistake is copying the wrong key.

Supabase displays several different keys.

Only the Publishable Key should be used in this workbook.

Never use the Service Role Key.

Never use a Secret Key.

Those keys are intended for secure server environments and should never appear inside files that visitors can download through their web browser.

Another common mistake is copying the REST API address instead of the Project URL.

Always use the base Project URL ending with:

.supabase.co

BEHIND THE SCENES

Whenever your application communicates with Supabase, it sends requests to your Project URL.

Those requests include your Publishable Key.

Supabase uses this information to identify your project before processing the request.

Although the Publishable Key is visible inside your website, it does not give visitors unrestricted access to your data.

Authentication and Row Level Security will provide that protection later in this workbook.

THINK LIKE A SOFTWARE DESIGNER

Professional software separates public information from private information.

The Project URL and Publishable Key are intended to be used by browser-based applications.

Sensitive credentials remain on secure servers and are never exposed to visitors.

Understanding this difference is one of the foundations of building secure software.

WHAT YOU LEARNED

In this lesson you learned how to:

- locate the Project URL
- locate the Publishable Key
- understand the purpose of both values
- avoid using secret keys
- prepare your project for the shared Supabase connection

In the next lesson, you will create a reusable Supabase connection file that every page in your Professional Invoice Generator will use.

CHAPTER 2

CONNECTING TO SUPABASE

LESSON 3

CREATING YOUR SUPABASE CONNECTION FILE

Estimated Time

30 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create a reusable JavaScript file that connects your Professional Invoice Generator to your Supabase project.

Rather than writing the connection code on every page, you will create one shared file named:

`supabase-config.js`

Every page that needs to communicate with Supabase will use this file.

This keeps your project organised and makes future updates much easier.

WHY THIS MATTERS

As software grows, several pages often need to communicate with the same database.

Instead of copying the connection code into every JavaScript file, professional developers create one shared connection file.

Whenever another page needs Supabase, it simply loads this file.

This approach reduces mistakes, keeps the project easier to maintain, and ensures every page uses exactly the same connection.

BEFORE YOU CONTINUE

Before starting this lesson, confirm that:

- ☒ Your Supabase project has been created.

✓ You have copied your Project URL.

✓ You have copied your Publishable Key.

Your Professional Invoice Generator folder should currently contain:

index.html

styles.css

script.js

You are about to add a fourth file.

BUILD PROMPT

PROMPT STARTS HERE

Create one complete JavaScript file named:

supabase-config.js

Return only the complete file.

Do not return explanations.

Do not return snippets.

Do not return HTML.

Do not return CSS.

GENERAL REQUIREMENTS

Create a reusable Supabase connection file.

Use the official Supabase browser client.

Create the client using this format exactly:

```
const supabaseClient = window.supabase.createClient(  
  "https://your-project-id.supabase.co",  
  "YOUR_PUBLISHABLE_KEY"  
);
```

Replace the placeholder values with my actual values.

Project URL:

PASTE YOUR PROJECT URL HERE

Publishable Key:

PASTE YOUR PUBLISHABLE KEY HERE

REQUIREMENTS

- Create only one Supabase client.
- Store it in a constant named:
`supabaseClient`
- Do not use:
`/rest/v1/`
inside the Project URL.
- Do not use:
Service Role Key
- Do not use:
Secret Key
- Do not use:
Database password
- Include a short comment at the top of the file explaining that this file creates the shared Supabase connection used throughout the application.
- Return one complete JavaScript file only.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return one complete file:

`supabase-config.js`

The file should:

- ☒ Create one shared Supabase client.
- ☒ Use my Project URL.

☒ Use my Publishable Key.

☒ Create the client using:

```
const supabaseClient = window.supabase.createClient(...)
```

☒ Contain JavaScript only.

SAVE YOUR FILES

Copy the complete JavaScript returned by ChatGPT.

Open Notepad.

Paste the code into a new empty document.

Click File.

Click Save As.

Open your Professional Invoice Generator folder.

Choose:

Save as type:

All Files

Enter the filename exactly as:

supabase-config.js

Click Save.

Close Notepad.

Your project folder should now contain:

index.html

styles.css

script.js

supabase-config.js

Make sure the file is not named:

supabase-config.js.txt

TEST YOUR WORK

Open your Professional Invoice Generator folder.

Confirm that:

supabase-config.js

appears alongside the other project files.

Open the file in Notepad.

Confirm that:

✓ The Project URL ends with:

.supabase.co

✓ The Project URL does not contain:

/rest/v1/

✓ The Publishable Key appears.

✓ The Supabase client is created using:

```
const supabaseClient = window.supabase.createClient(...)
```

At this stage, the connection file is not yet being used by any webpage.

That is expected.

The connection will be tested in the next lesson.

CHECKPOINT

Before moving on, confirm that:

✓ supabase-config.js has been created.

✓ The filename is correct.

✓ The Project URL is correct.

✓ The Publishable Key is correct.

✓ Only one Supabase client exists.

COMMON BEGINNER MISTAKES

One common mistake is copying the REST API address instead of the Project URL.

Always use the base URL ending with:

.supabase.co

Another common mistake is copying the Service Role Key instead of the Publishable Key.

Only the Publishable Key belongs inside this file.

Never place secret keys inside browser-based applications.

Some beginners also create another Supabase client later inside `register.js` or `dashboard.js`.

Do not do that.

Throughout this workbook, every page will use the single shared client created inside:

`supabase-config.js`

BEHIND THE SCENES

The line:

```
const supabaseClient = window.supabase.createClient(...)
```

creates one reusable connection between your application and your Supabase project.

Later in this workbook, every page that needs Supabase will simply use:

`supabaseClient`

instead of creating another connection.

This keeps the application consistent and greatly reduces the chance of connection errors.

THINK LIKE A SOFTWARE DESIGNER

Whenever several parts of an application need the same functionality, it is usually better to create that functionality once and reuse it everywhere.

This principle keeps software simpler, easier to maintain, and less likely to contain conflicting code.

Creating a shared Supabase connection file is one example of that idea.

CODE-READING QUESTION

Open supabase-config.js. Which line creates the Supabase client, and which two saved values does that line use?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a shared Supabase connection file
- create one reusable Supabase client
- use the correct Project URL
- use the correct Publishable Key
- avoid exposing secret keys
- prepare your application for future database communication

In the next lesson, you will build a connection test page that confirms your Professional Invoice Generator can communicate successfully with your Supabase project.

CHAPTER 2

CONNECTING TO SUPABASE

LESSON 4

BUILDING A CONNECTION TEST PAGE

Estimated Time

35 minutes

WHAT YOU ARE BUILDING

In this lesson, you will build a simple webpage that confirms your Professional Invoice Generator can communicate successfully with your Supabase project.

This page exists only for testing.

It is not part of the finished application.

Instead, it allows you to verify that your shared Supabase connection is working correctly before you begin building authentication.

WHY THIS MATTERS

Professional developers test important parts of an application before building additional features.

Imagine spending hours creating registration and login pages only to discover that your website was never connected to Supabase correctly.

Testing the connection now helps you identify problems while the project is still simple.

BEFORE YOU CONTINUE

Your project folder should currently contain:

index.html

styles.css

script.js

supabase-config.js

You should also have:

- ☒ Your Supabase project
- ☒ Your Project URL
- ☒ Your Publishable Key

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have these files:

index.html

styles.css

script.js

supabase-config.js

Everything already built must continue working.

Do not modify any existing files.

Create one new HTML file named:

test-connection.html

Return one complete HTML file only.

Do not return explanations.

Do not return snippets.

GENERAL REQUIREMENTS

Create a complete HTML5 document.

Include:

- Appropriate page title
- Viewport meta tag

Inside the head section, include simple CSS that creates a clean, centred layout suitable for a testing page.

BODY CONTENT

Display:

Heading

Supabase Connection Test

Below the heading, display a short paragraph explaining that the page is checking communication with the Supabase project.

Create a status box.

The status box should initially display:

Checking connection...

Give this element the ID:

connection-status

SCRIPT LOADING ORDER

Immediately before the closing body tag, load these scripts in this exact order.

First:

```
<script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2"></script>
```

Second:

```
<script src="supabase-config.js"></script>
```

Third:

Create an inline JavaScript block that:

- Checks whether:

supabaseClient

exists.

If it exists:

Change the status box to display:

✅ Connected successfully to Supabase.

Apply a success style.

If it does not exist:

Display:

❌ Unable to connect to Supabase.

Apply an error style.

IMPORTANT

Use the existing:

`supabase-config.js`

Do not create another Supabase client.

Do not hard-code the Project URL.

Do not hard-code the Publishable Key.

Do not use authentication.

Do not create database tables.

Do not create users.

Return one complete HTML file only.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return one complete file:

`test-connection.html`

The page should:

- ☒ Load the Supabase browser library.
- ☒ Load `supabase-config.js`.
- ☒ Load scripts in the correct order.
- ☒ Check whether `supabaseClient` exists.
- ☒ Display a clear success or error message.

SAVE YOUR FILES

Copy the complete HTML returned by ChatGPT.

Open Notepad.

Paste the code into a new document.

Click File.

Click Save As.

Open your Professional Invoice Generator folder.

Choose:

Save as type:

All Files

Enter the filename exactly as:

test-connection.html

Click Save.

Close Notepad.

Your project folder should now contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

TEST YOUR WORK

Open:

test-connection.html

The page should open in your browser.

After a moment, the message should change from:

Checking connection...

to:

✅ Connected successfully to Supabase.

If instead you see:

❌ Unable to connect to Supabase.

Do not continue.

Return to the previous lessons and carefully check:

- ✓ Project URL
- ✓ Publishable Key
- ✓ supabase-config.js
- ✓ Script loading order

CHECKPOINT

Before moving on, confirm that:

- ✓ test-connection.html opens.
- ✓ The page loads correctly.
- ✓ The Supabase browser library loads first.
- ✓ supabase-config.js loads second.
- ✓ The page displays:

Connected successfully to Supabase.

COMMON BEGINNER MISTAKES

The most common mistake is loading the JavaScript files in the wrong order.

The correct order is always:

1. Supabase browser library
2. supabase-config.js
3. Page-specific JavaScript

Another common mistake is creating another Supabase client inside the page.

Do not do this.

Your application should use only the shared client created inside:

supabase-config.js

BEHIND THE SCENES

This page does not create a new connection.

Instead, it loads the shared connection file.

If that file creates:

`supabaseClient`

successfully, the page simply confirms that the shared connection is available.

Every protected page you build later will use this same approach.

THINK LIKE A SOFTWARE DESIGNER

Professional software is built on reliable foundations.

Testing one important component before moving to the next makes larger projects much easier to debug.

Finding problems early almost always saves time later.

CODE-READING QUESTION

Open `test-connection.html`. Find the part of the page added for building a connection test page. Which HTML element starts that part?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- build a dedicated connection test page
- load the Supabase browser library
- reuse a shared connection file
- verify that your application can communicate with Supabase
- identify common connection problems

CHAPTER 2

CONNECTING TO SUPABASE

LESSON 5

FINAL CONNECTION REVIEW

Estimated Time

15 minutes

WHAT YOU ARE BUILDING

This lesson is your final review before building the authentication system.

You are not creating any new files.

Instead, you will carefully inspect your project and confirm that every connection to Supabase has been configured correctly.

WHY THIS MATTERS

The authentication system depends completely on your Supabase connection.

If the connection is incorrect, registration, login, invoice management and every future database feature will fail.

Confirming your work now helps prevent much larger problems later.

BEFORE YOU CONTINUE

Your project folder should now contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, review your project carefully.

Complete the following checks.

Open:

supabase-config.js

Confirm that:

☒ The Project URL ends with:

.supabase.co

☒ The Project URL does not contain:

/rest/v1/

☒ The Publishable Key is being used.

☒ The shared client is created using:

```
const supabaseClient = window.supabase.createClient(...)
```

Next, open:

test-connection.html

Confirm that the scripts load in this exact order:

1.

Supabase browser library

2.

supabase-config.js

3.

The page's own JavaScript

Finally, open:

test-connection.html

Confirm that the page displays:

☒ Connected successfully to Supabase.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson is completed by reviewing your project.

SAVE YOUR FILES

No files are created or updated during this lesson.

TEST YOUR WORK

Complete this checklist.

Landing Page

☒ index.html opens correctly.

☒ Navigation works.

☒ Responsive menu works.

Supabase

☒ Project created.

☒ Project URL correct.

☒ Publishable Key correct.

☒ Shared connection file created.

Connection Test

☒ test-connection.html opens.

☒ Connected successfully to Supabase appears.

Project Folder

☒ index.html

☒ styles.css

☒ script.js

✓ supabase-config.js

✓ test-connection.html

CHECKPOINT

Before moving to Chapter 3, confirm that:

✓ Every required file exists.

✓ The landing page still works.

✓ The connection test succeeds.

✓ The Project URL is correct.

✓ The Publishable Key is correct.

✓ The script loading order is correct.

✓ Only one Supabase client exists.

COMMON BEGINNER MISTAKES

Some students continue building after the connection test has failed.

Do not do this.

Authentication depends on the connection working correctly.

Solve connection problems now before moving to the next chapter.

BEHIND THE SCENES

At this point, your Professional Invoice Generator knows how to communicate with Supabase.

What it cannot do yet is identify users.

That is the purpose of the next chapter.

Authentication allows the application to recognise who is currently signed in so every invoice can be linked to the correct person.

THINK LIKE A SOFTWARE DESIGNER

Every large software project is built in layers.

Public website.

Backend connection.

Authentication.

Database.

Business features.

Building each layer carefully creates a much more reliable application than trying to build everything at once.

WHAT YOU LEARNED

In this lesson you learned how to:

- verify the shared Supabase connection
- review script loading order
- confirm that your application is ready for authentication
- understand why authentication comes before invoice management

CHAPTER SUMMARY

Congratulations.

Your Professional Invoice Generator is now successfully connected to Supabase.

You created your own Supabase project, located the Project URL and Publishable Key, built a shared connection file, and confirmed that your application can communicate successfully with your online backend.

More importantly, you established a reusable architecture that every future page in the application will use.

Whenever a page needs Supabase, it will always load:

1. The Supabase browser library.
2. `supabase-config.js`.
3. The page-specific JavaScript file.

Maintaining this structure throughout the project will keep your application organised and reduce connection-related problems.

CHAPTER MILESTONE

By the end of Chapter 2, you have successfully completed:

- ✓ Supabase project
- ✓ Project URL
- ✓ Publishable Key
- ✓ Shared connection file
- ✓ Connection test page
- ✓ Connection verification
- ✓ Correct script loading order
- ✓ Backend preparation for authentication

Your project folder should now contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

TRANSITION TO CHAPTER 3

Your public website is complete.

Your application is connected to Supabase.

The next step is to build the complete authentication system.

In Chapter 3, you will build registration, login, the protected dashboard, and the protected invoice details page as one coordinated system.

By creating every destination page together, every redirect will already have a valid destination, allowing the complete authentication workflow to function correctly from the very beginning.

CHAPTER 3

BUILDING THE COMPLETE AUTHENTICATION SYSTEM

CHAPTER INTRODUCTION

Your Professional Invoice Generator now has a professional public website and a working connection to Supabase.

The next step is to allow people to create their own accounts and access their own invoices securely.

Authentication is one of the most important parts of any business application.

Without authentication, every visitor would have access to the same invoice database.

That would make the software unsafe to use.

In this chapter, you will build the complete authentication system as one coordinated feature.

Instead of creating one page at a time, you will create every authentication page together.

This approach ensures that:

- Every redirect already has a destination.
- Every page is correctly linked.
- Every JavaScript file loads correctly.
- Every stylesheet is linked correctly.
- The complete user journey works from the very beginning.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- register.html
- login.html

- dashboard.html
- invoice-details.html
- auth.css
- register.js
- login.js
- dashboard.js
- invoice-details.js

You will also update:

- index.html

The completed authentication flow will work like this:

Visitor

↓

index.html

↓

Register

↓

register.html

↓

Email verification

↓

login.html

↓

Login

↓

dashboard.html

↓

Invoice Details

↓

invoice-details.html

↓

Logout

↓

login.html

Unauthenticated visitors who attempt to open:

dashboard.html

or

invoice-details.html

will be redirected automatically to:

login.html

LESSON 1

UNDERSTANDING THE AUTHENTICATION FLOW

Estimated Time

15 minutes

WHAT YOU ARE BUILDING

Before creating the authentication system, it is important to understand how every page works together.

Authentication is not simply a login page.

It is a complete journey that begins when a visitor creates an account and continues until they safely sign out.

Every page depends on another page.

That is why the entire system will be built together.

WHY THIS MATTERS

Many beginners build authentication one page at a time.

The result is often broken redirects, missing files and incomplete user journeys.

Professional developers think about the complete workflow before writing any code.

Doing so reduces mistakes and produces a much more reliable application.

BEFORE YOU CONTINUE

Your project folder should currently contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

Confirm that:

- ✓ The landing page works.
- ✓ The Supabase connection test succeeds.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, study the authentication flow below.

A visitor arrives at:

index.html

The visitor selects:

Register

The visitor opens:

register.html

The visitor creates an account.

Supabase sends a verification email.

The visitor verifies the email address.

The visitor returns to:

login.html

The visitor signs in.

The application redirects to:

dashboard.html

The user manages invoices.

Whenever the user opens an invoice details, the application opens:

invoice-details.html

When the user signs out, the application redirects to:

login.html

If the visitor is not authenticated and tries to open:

dashboard.html

or

invoice-details.html

the application immediately redirects them to:

```
login.html
```

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No files are created during this lesson.

TEST YOUR WORK

Review the authentication flow.

Make sure you understand where every page sends the user.

Every destination page already has a purpose.

CHECKPOINT

Before moving on, confirm that you understand:

- ☒ Registration comes before Login.
- ☒ Login comes before Dashboard.
- ☒ Invoice Details is protected.
- ☒ Logout returns the user to login.html.
- ☒ Protected pages require authentication.

COMMON BEGINNER MISTAKES

One common mistake is thinking that a login page alone creates a complete authentication system.

A secure authentication system includes:

- Registration
- Email verification

- Login
- Protected pages
- Logout
- Automatic redirects

All of these parts work together.

BEHIND THE SCENES

Authentication allows the application to answer one important question:

Who is currently using the software?

Once that question has been answered, every invoice can be connected to the correct user.

THINK LIKE A SOFTWARE DESIGNER

Always think about the complete journey.

Instead of asking:

"What happens on this page?"

ask:

"Where did the user come from?"

and

"Where should the user go next?"

That way, every page becomes part of one connected application.

WHAT YOU LEARNED

In this lesson you learned:

- the complete authentication workflow
- how every authentication page connects together
- why protected pages exist
- why redirects are planned before building begins

CHAPTER 3

BUILDING THE COMPLETE AUTHENTICATION SYSTEM

LESSON 2

BUILDING THE COMPLETE AUTHENTICATION SYSTEM

Estimated Time

75 minutes

WHAT YOU ARE BUILDING

In this lesson, you will build the complete authentication system for your Professional Invoice Generator.

Rather than generating one page at a time, ChatGPT will generate every authentication page together.

This ensures that:

- Every destination page already exists.
- Every redirect already has somewhere to go.
- Every stylesheet is linked correctly.
- Every JavaScript file is loaded correctly.
- Everything already built continues working.

WHY THIS MATTERS

Registration, login, dashboard access and invoice details are tightly connected.

Building them together reduces missing files, broken redirects and inconsistent behaviour.

This coordinated approach reduces missing files and broken redirects.

BEFORE YOU CONTINUE

Your project folder should currently contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

Confirm that:

☒ The connection test displays:

Connected successfully to Supabase.

Do not continue until everything above is working.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Professional Invoice Generator.

Everything already built must continue working.

My project currently contains:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

The public website is complete.

The responsive navigation works.

The Supabase connection has been tested successfully.

Do not remove or break any existing functionality.

Return complete files only.

Do not return snippets.

Do not return explanations.

Return the following complete files:

1.

Updated index.html

2.

register.html

3.

login.html

4.

dashboard.html

5.

invoice-details.html

6.

auth.css

7.

register.js

8.

login.js

9.

dashboard.js

10.

invoice-details.js

GENERAL REQUIREMENTS

Everything already built must continue working.

Every HTML page must be complete.

Every HTML page must include proper HTML5 structure.

Every HTML page must include an appropriate page title.

Use semantic HTML.

Use modern, responsive layouts.

The logo on every page should display:

Invoice Records

The logo should link to:

index.html

INDEX.HTML

Return a complete updated version.

Preserve:

- Hero
- Features
- Navigation
- Footer
- Responsive menu

Ensure Login points to:

login.html

Ensure Register points to:

register.html

REGISTER.HTML

Create a professional registration page.

Include:

- Logo
- Full Name
- Email Address
- Password
- Confirm Password
- Register button
- Link to login.html

Display friendly validation messages.

Disable the Register button while processing.

Display loading messages.

After successful registration:

Display a success message explaining that the user must verify their email before signing in.

Redirect the user to:

login.html

Do not automatically sign the user in.

LOGIN.HTML

Create a professional login page.

Include:

- Logo
- Email
- Password
- Login button
- Link to register.html

Display friendly validation and loading messages.

Disable the Login button while processing.

If login succeeds:

Redirect to:

dashboard.html

If the user is already authenticated:

Automatically redirect to:

dashboard.html

DASHBOARD.HTML

Create a professional dashboard page.

Include:

- Logo
- Welcome heading
- Logged-in user's email
- Four invoice summary cards with initial values of 0:

Total Invoices

Draft Invoices

Sent Invoices

Paid Invoices

Include a placeholder section where the invoice management dashboard will be built later.

Include:

Logout button

If the visitor is not authenticated:

Redirect immediately to:

login.html

INVOICE-DETAILS.HTML

Create a protected invoice details page.

Include:

- Logo
- Invoice details heading
- Placeholder invoice information
- Back to Dashboard button
- Logout button

If the visitor is not authenticated:

Redirect immediately to:

login.html

AUTH.CSS

Create one shared stylesheet for:

register.html

login.html

dashboard.html

invoice-details.html

Use:

Professional forms

Responsive layout

Business dashboard styling

Summary cards

Loading messages

Success messages

Error messages

REGISTER.JS

Load after:

1. Supabase CDN
2. supabase-config.js

Use:

```
const supabaseClient
```

already created inside:

```
supabase-config.js
```

Do not create another Supabase client.

Validate:

- Full Name
- Email
- Password
- Confirm Password

Passwords must match.

Create the account using Supabase Authentication.

When calling:

```
supabaseClient.auth.signUp
```

include an email confirmation destination.

Create it from the website that is currently open:

```
`${window.location.origin}/login.html`
```

Pass that address as:

```
emailRedirectTo
```

inside the sign-up options.

Do not hardcode a Netlify website name.

Using:

```
window.location.origin
```

allows the same complete files to work when the website address changes.

Display friendly success and error messages.

Disable the button while processing.

After successful registration:

Inform the user to open the verification email.

Redirect to:

login.html

Do not automatically sign the user in from the registration form.

LOGIN.JS

Load after:

1. Supabase CDN

2. supabase-config.js

Use:

supabaseClient

Authenticate users.

Display loading messages.

Display friendly errors.

Redirect successful logins to:

dashboard.html

DASHBOARD.JS

Load after:

1. Supabase CDN

2. supabase-config.js

Use:

supabaseClient

Confirm that the user is authenticated.

Display the authenticated user's email.

Support Logout.

Redirect unauthenticated users to:

login.html

INVOICE-DETAILS.JS

Load after:

1. Supabase CDN

2. supabase-config.js

Use:

supabaseClient

Confirm that the visitor is authenticated.

If not:

Redirect immediately to:

login.html

Support Logout.

Invoices will be loaded later in this workbook.

For now, create the protected page and its authentication logic.

SCRIPT LOADING ORDER

Every Supabase-powered HTML page must load:

1.

Supabase browser library

2.

supabase-config.js

3.

Its own page-specific JavaScript

IMPORTANT

Return every requested file completely.

Do not omit any file.

Do not return partial updates.

Everything already built must continue working.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return ten complete files.

Every HTML page should already contain the correct stylesheet and JavaScript links.

Every protected page should already verify authentication.

Every redirect should already point to an existing page.

CHAPTER 3

BUILDING THE COMPLETE AUTHENTICATION SYSTEM

LESSON 2 (CONTINUED)

BUILDING THE COMPLETE AUTHENTICATION SYSTEM

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

ChatGPT should return ten complete files.

Work through them one at a time.

If the file already exists, replace its complete contents.

If the file does not already exist, create it.

Do not copy only part of a file.

Always replace the complete contents.

Updated index.html

Open:

index.html

in Notepad.

Select all the existing code.

Delete it.

Paste the complete updated code returned by ChatGPT.

Click:

File

Save

Close Notepad.

register.html

Open Notepad.

Create a new document.

Paste the complete code.

Click:

File

Save As

Open your Professional Invoice Generator folder.

Choose:

Save as type:

All Files

Enter:

register.html

Click Save.

login.html

Repeat the same process.

Save as:

login.html

dashboard.html

Repeat the same process.

Save as:

dashboard.html

invoice-details.html

Repeat the same process.

Save as:

invoice-details.html

auth.css

Repeat the same process.

Save as:

auth.css

register.js

Repeat the same process.

Save as:

register.js

login.js

Repeat the same process.

Save as:

login.js

dashboard.js

Repeat the same process.

Save as:

dashboard.js

invoice-details.js

Repeat the same process.

Save as:

invoice-details.js

When you finish, your project folder should contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

register.html

login.html

dashboard.html

invoice-details.html

auth.css

register.js

login.js

dashboard.js

invoice-details.js

Check every filename carefully.

Do not allow any file to become:

register.html.txt

login.js.txt

dashboard.html.txt

or any similar variation.

TEST YOUR WORK

Do not assume the authentication system works.

Authentication must be tested from a website with an HTTPS address.

Do not test this chapter by double-clicking the HTML files in your project folder.

The files still remain inside your project folder.

You will simply place a test copy of the complete folder online before testing.

BEFORE YOU START THE TESTS

1.

Save every updated file in Notepad.

2.

Deploy the complete project folder to Netlify.

If you already created a Netlify test website for this project, deploy the updated folder to the same website.

3.

Copy the website address supplied by Netlify.

It should begin with:

`https://`

For example:

`https://your-site-name.netlify.app`

Your own website address will be different.

4.

Open your Supabase project.

Go to:

Authentication

Then open:

URL Configuration

5.

Set the Site URL to your Netlify website address.

Do not add a file name to the Site URL.

6.

Add this complete address to Redirect URLs:

`https://your-site-name.netlify.app/login.html`

Replace the example website name with your real Netlify website name.

7.

Make sure the latest version of every file has been deployed.

8.

Open the project from its Netlify HTTPS address.

Keep using that address throughout every test below.

TEST 1

PUBLIC WEBSITE

Open the deployed index.html page.

Confirm that:

- ☒ The landing page opens correctly.
- ☒ The page is fully styled.
- ☒ The responsive navigation still works.
- ☒ Login opens the deployed login.html page.
- ☒ Register opens the deployed register.html page.

Look at the browser address bar.

The address must begin with:

https://

Nothing already built should have stopped working.

TEST 2

REGISTRATION PAGE

Open the deployed register.html page.

Attempt to submit the form without entering any information.

Friendly validation messages should appear.

Complete the form using an email address that has not already been used for this test.

Click:

Register

The Register button should become disabled while the request is being processed.

A loading message should appear.

After successful registration:

You should see a message asking you to verify your email.

TEST 3

EMAIL VERIFICATION

Open your email inbox.

Locate the verification email from Supabase.

Open the email.

Click the verification link.

Supabase should verify the email and return the browser to your deployed website.

The returned address should use your Netlify website name.

It should not begin with:

file://

Depending on the authentication response, you may briefly see login.html or the application may recognise the new session and open the dashboard.

Both results confirm that the browser returned to the correct website.

If the login page remains open, sign in using the email address and password you registered.

If the verification email does not arrive immediately:

Wait a few minutes.

Refresh your inbox.

Check your Spam or Junk folder.

If the verification link opens the wrong website:

Return to Supabase URL Configuration.

Check the Site URL and Redirect URLs carefully.

Correct them before creating another test account.

TEST 4

LOGIN

Open the deployed login.html page.

Enter an incorrect password.

A friendly error message should appear.

Now enter the correct password.

Click:

Login

The Login button should become disabled while signing in.

A loading message should appear.

After a successful login:

The browser should open the deployed dashboard.html page.

TEST 5

AUTHENTICATED SESSION

After signing in successfully:

Confirm that the protected dashboard opens.

Confirm that your email address appears.

Refresh the page.

The dashboard should remain open because the same website still has your authenticated session.

Confirm that the four invoice summary cards appear.

Confirm that the Logout button appears.

While signed in, open the complete deployed invoice-details.html address.

The protected invoice details page should open.

Click:

Logout

The browser should return to the deployed login.html page.

While signed out, open the complete deployed dashboard.html address.

The application should immediately return you to the deployed login.html page.

While still signed out, open the complete deployed invoice-details.html address.

The application should immediately return you to the deployed login.html page.

Sign in again.

While signed in, open the complete deployed login.html address.

The application should immediately return you to the deployed dashboard.html page.

CHECKPOINT

Before moving to the next lesson, confirm that:

- ☒ Registration works.
- ☒ Validation works.
- ☒ Loading messages appear.
- ☒ Email verification succeeds.
- ☒ Login works.
- ☒ Friendly error messages appear.
- ☒ Dashboard is protected.
- ☒ Invoice Details page is protected.
- ☒ Logout works.
- ☒ Authenticated users cannot remain on login.html.
- ☒ Unauthenticated users cannot open dashboard.html.
- ☒ Unauthenticated users cannot open invoice-details.html.

COMMON BEGINNER MISTAKES

One common mistake is forgetting to verify the email address before attempting to sign in.

If email verification is enabled, Supabase will not allow the account to be used until verification has been completed.

Another common mistake is loading the JavaScript files in the wrong order.

Every Supabase-powered page should load:

1.

Supabase browser library

2.

supabase-config.js

3.

The page-specific JavaScript

Changing this order can prevent the application from working correctly.

Some beginners also create another Supabase client inside:

register.js

login.js

dashboard.js

or

invoice-details.js

Do not do this.

Every page should use the shared client created inside:

supabase-config.js

BEHIND THE SCENES

Your authentication system is now working as one connected application.

When a visitor creates an account, Supabase stores the authentication information securely.

After email verification, Supabase allows that account to sign in.

When login succeeds, Supabase creates a session.

Protected pages check whether that session exists before displaying their content.

If the session does not exist, the visitor is redirected immediately to the login page.

The Invoice Details page already uses this protection even though invoices have not yet been introduced.

Later, this same authentication system will protect every invoice in the application.

THINK LIKE A SOFTWARE DESIGNER

Authentication is more than allowing someone to sign in.

It controls who can enter the application, which pages they can open, and what information they are allowed to see.

Every protected page should perform its own authentication check.

Never assume that because a user reached one protected page, every other page is automatically secure.

CODE-READING QUESTION

Open `register.js`. Find one function that supports building the complete authentication system. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- build a complete authentication system
- connect registration, login and protected pages
- protect multiple destinations
- redirect unauthenticated visitors
- redirect authenticated users appropriately
- preserve every feature already built

CHAPTER 3

BUILDING THE COMPLETE AUTHENTICATION SYSTEM

LESSON 3

AUDITING THE AUTHENTICATION SYSTEM

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will not build new features.

Instead, you will carefully review every part of the authentication system before beginning invoice management.

Authentication is the foundation of the entire application.

Every invoice created later will depend on the current user being identified correctly.

WHY THIS MATTERS

The next chapter introduces invoices.

Every invoice must belong to one authenticated user.

If authentication is unreliable, invoice privacy cannot be guaranteed.

Professional developers verify security before adding business features.

BEFORE YOU CONTINUE

Your project folder should now contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

register.html

login.html

dashboard.html

invoice-details.html

auth.css

register.js

login.js

dashboard.js

invoice-details.js

Confirm that you have successfully created at least one verified user account.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Complete every authentication test again.

Do not continue until every test succeeds.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No files are created or updated during this lesson.

TEST YOUR WORK

Complete this final authentication check on the deployed HTTPS website.

Do not complete this audit from files opened with:

file://

SUPABASE URL SETTINGS

- ✓ The Site URL contains the Netlify website address.
- ✓ Redirect URLs contains the complete deployed login.html address.

REGISTRATION AND VERIFICATION

- ✓ Registration works from the deployed register.html page.
- ✓ A verification email arrives.
- ✓ The verification link returns to the deployed website.
- ✓ The returned address begins with https:// and uses the correct Netlify website name.

LOGIN AND SESSION

- ✓ Login works from the deployed login.html page.
- ✓ Refreshing a protected page keeps a signed-in user signed in.
- ✓ Logout works.

PROTECTION

- ✓ A signed-out visitor cannot remain on dashboard.html.
- ✓ A signed-in user can reach dashboard.html.
- ✓ A signed-in user who opens login.html returns to dashboard.html.

PROTECTED INVOICE PAGE

- ✓ A signed-in user can open invoice-details.html.
- ✓ A signed-out visitor cannot remain on invoice-details.html.

PAGES

- ✓ The signed-in user's email appears.
- ✓ The summary cards appear.

- ✓ The invoice details placeholder appears.

If every check passes, the authentication system is ready for the next chapter.

CHECKPOINT

Before moving to Chapter 4, confirm that:

- ✓ Every authentication page works.
- ✓ Every redirect works.
- ✓ Dashboard protection works.
- ✓ Invoice Details protection works.
- ✓ No authentication errors remain.

COMMON BEGINNER MISTAKES

Some students test only the successful login process.

Always test unsuccessful situations as well.

Protected pages should remain protected when no user is signed in.

Another common mistake is assuming one protected page automatically secures every other page.

Each protected page should verify authentication independently.

BEHIND THE SCENES

The authentication system now provides the identity that every future database operation will rely on.

When invoices are introduced in Chapter 4, every new invoice will automatically belong to the currently authenticated user.

Authentication and Row Level Security will work together to protect invoice information.

THINK LIKE A SOFTWARE DESIGNER

Security should be confirmed before adding business functionality.

A reliable authentication system makes every future feature easier to build because the application always knows who the current user is.

WHAT YOU LEARNED

In this lesson you learned how to:

- audit an authentication system
- verify protected pages
- verify redirects
- prepare the application for secure invoice management

CHAPTER SUMMARY

Congratulations.

Your Professional Invoice Generator now includes a complete authentication system.

Visitors can register, verify their email addresses, sign in, access protected pages, and sign out securely.

Every protected page already knows how to verify the current user's session before displaying information.

Most importantly, your application can now identify who is currently using the system.

That capability will allow every invoice to belong to the correct user.

CHAPTER MILESTONE

By the end of Chapter 3, you have successfully built:

- ✓ User registration
- ✓ Email verification
- ✓ Secure login
- ✓ Protected dashboard
- ✓ Protected invoice details page
- ✓ Logout
- ✓ Friendly validation
- ✓ Friendly success messages
- ✓ Friendly error messages
- ✓ Loading states
- ✓ Shared authentication styling
- ✓ Complete authentication workflow

TRANSITION TO CHAPTER 4

Your Professional Invoice Generator can now identify every authenticated user.

The next step is to create the secure invoice database.

In Chapter 4, you will design the invoices table, create a richer business data model, enable Row Level Security, and prepare the application to store invoice information safely.

This database structure allows businesses to manage complete invoices containing several related items.

CHAPTER 4

BUILDING THE INVOICE DATABASE

CHAPTER INTRODUCTION

Your Professional Invoice Generator can now recognise the signed-in user.

The next step is to create a secure database for customers, invoices and invoice items.

The customers table will store reusable customer profiles.

The invoices table will store each complete business document and a snapshot of the selected customer.

The invoice_items table will store every item on each invoice.

You will add database constraints and Row Level Security so invalid or unauthorised information is rejected before it can become trusted business data.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- customers table
- invoices table
- invoice_items table
- Customer and invoice relationships
- Invoice item cascading deletion
- Database constraints
- Row Level Security on all three tables
- SELECT, INSERT, UPDATE and DELETE policies
- Two-account privacy tests

LESSON 1

UNDERSTANDING THE INVOICE DATA MODEL

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

Before creating the database, you will understand how the information is organised.

The application uses three related tables.

The customers table stores reusable customer profiles.

The invoices table stores each complete invoice and a snapshot of the selected customer.

The invoice_items table stores the individual lines that belong to each invoice.

This lesson explains why all three tables are needed.

WHY THIS MATTERS

Good software begins with good data design.

If the database is designed poorly, the software becomes difficult to improve later.

Professional developers spend time deciding what information should be stored before creating the database itself.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Registration works.
- ☒ Login works.
- ☒ Dashboard protection works.
- ☒ Your Supabase project is open.

No changes will be made to your project files during this lesson.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Study the database plan below.

CUSTOMERS TABLE

This table stores:

- The signed-in user's ID
- Customer name
- Company name
- Email
- Phone
- Billing address

INVOICES TABLE

This table stores:

- The signed-in user's ID
- The selected customer ID
- A unique invoice number
- Business details
- A snapshot of the selected customer details
- Issue and due dates
- Invoice status and currency
- Discount, tax and trusted totals
- Notes and dates

INVOICE_ITEMS TABLE

This table stores:

- The parent invoice ID
- Description
- Quantity

- Unit price
 - Trusted line total
 - Item position
- One customer can be selected for several invoices.
- One invoice can contain several invoice items.
- Deleting an invoice should delete its invoice items.
- Deleting a customer with saved invoices should not rewrite or remove those invoices.
- Row Level Security will protect all three tables.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No files are created during this lesson.

TEST YOUR WORK

- Review the complete invoice data model.
- Ask yourself:
- Which fields are required?
 - Which fields are optional?
 - Why should a saved invoice keep a customer snapshot?
- If you can answer these questions, you are ready for the next lesson.

CHECKPOINT

Before moving on, confirm that you understand:

- ☒ Required fields
- ☒ Optional fields

- ✓ Why invoices contain richer information than expense records

COMMON BEGINNER MISTAKES

Some beginners make every field required.

This often frustrates users because they may not know every piece of information when creating a new invoice.

Another common mistake is collecting too little information.

An invoice database should contain enough detail to make future communication easier.

BEHIND THE SCENES

This database has been designed to balance simplicity with practical business use.

It collects enough information to support invoice management without becoming unnecessarily complicated.

Later workbooks will build on these same design principles.

THINK LIKE A SOFTWARE DESIGNER

Every database field should have a purpose.

Before adding a new field, ask yourself:

"Will this information help the business make better decisions or provide better service?"

If the answer is no, the field may not belong in the database.

WHAT YOU LEARNED

In this lesson you learned:

- how invoice information is organised
- the difference between required and optional fields
- why database design matters
- how business software stores richer information

CHAPTER 4

BUILDING THE INVOICE DATABASE

LESSON 2

CREATING THE CUSTOMERS TABLE

Estimated Time

30 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create the table that stores customers.

The invoice form will later load these saved customers into a simple selection list.

Every customer will belong to the user who created that customer.

WHY THIS MATTERS

Businesses often create several invoices for the same customer.

Saving the customer once prevents the business from typing the same name, email address, phone number and address every time.

It also makes the invoice process faster and reduces typing mistakes.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Your Supabase project is open.
- ☒ Authentication works on the deployed HTTPS website.
- ☒ You understand why this application needs three related tables.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Create the customers table directly inside Supabase.

Open:

Table Editor

Click:

Create a new table

Name the table:

customers

Leave Row Level Security enabled.

Create these columns:

- id – uuid – Primary Key – default value gen_random_uuid()
- user_id – uuid – required
- customer_name – text – required
- company_name – text – optional
- email – text – optional
- phone – text – optional
- billing_address – text – optional
- created_at – timestamp with time zone – required – default value now()
- updated_at – timestamp with time zone – required – default value now()

Review every column.

Then click:

Save

Do not add invoice totals or invoice items to this table.

This table stores reusable customer information only.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson is completed inside Supabase.

SAVE YOUR FILES

No project files are created during this lesson.

TEST YOUR WORK

Open:

customers

inside the Table Editor.

Confirm that:

- ☒ id is the Primary Key.
- ☒ user_id exists and is required.
- ☒ customer_name exists and is required.
- ☒ company_name, email, phone and billing_address are optional.
- ☒ created_at and updated_at exist.
- ☒ Row Level Security remains enabled.

Do not add a customer row manually.

The application will add customers after the correct security policies exist.

CHECKPOINT

Before moving on, confirm that:

- ☒ The customers table exists.
- ☒ Every column has the correct type.
- ☒ Row Level Security remains enabled.

COMMON BEGINNER MISTAKES

A common mistake is making every contact field required.

A business may know a customer's phone number without knowing the email address.

Only the customer name is required in this workbook.

Another mistake is putting invoice information inside the customers table.

Customer information and invoice information have different jobs and must remain separate.

BEHIND THE SCENES

The customers table stores the current reusable customer profile.

Each saved invoice will also keep a copy of the customer details used when that invoice was created.

This means an old invoice does not change when the customer profile is updated later.

THINK LIKE A SOFTWARE DESIGNER

Business documents should preserve their history.

A customer profile may change, but a saved invoice should continue showing the information that was used when the invoice was issued.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a reusable customers table
- keep customer and invoice information separate
- prepare the application for customer selection
- preserve invoice history

CHAPTER 4

BUILDING THE INVOICE DATABASE

LESSON 3

CREATING THE INVOICES TABLE

Estimated Time

35 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create the database table that stores every invoice.

Each row inside the table will represent one invoice.

Every invoice will belong to one authenticated user.

Later, the dashboard will use this table to create, display, update and delete invoices.

WHY THIS MATTERS

Without a properly designed table, your application has nowhere to store invoice information.

Building the correct structure now makes every later feature much easier to develop.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Authentication is working.
- ☒ Your Supabase project is open.
- ☒ You understand the invoice data model.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Create the invoices table directly inside Supabase.

Open:

Table Editor

Create a new table named:

invoices

Leave Row Level Security enabled.

Create these columns:

- id – uuid – Primary Key – default value gen_random_uuid()
- user_id – uuid – required
- customer_id – uuid – required
- invoice_number – text – required
- business_name – text – required
- business_email – text – optional
- business_phone – text – optional
- business_address – text – optional
- customer_name – text – required
- customer_company – text – optional
- customer_email – text – optional
- customer_phone – text – optional
- customer_address – text – optional
- issue_date – date – required
- due_date – date – required
- status – text – required – default value draft
- currency – text – required – default value GBP
- discount_type – text – required – default value none
- discount_value – numeric – required – default value 0
- subtotal – numeric – required – default value 0
- discount_amount – numeric – required – default value 0
- tax_rate – numeric – required – default value 0
- tax_amount – numeric – required – default value 0
- total – numeric – required – default value 0
- notes – text – optional
- created_at – timestamp with time zone – required – default value now()

- `updated_at` – timestamp with time zone – required – default value `now()`

Create a relationship from:

`invoices.customer_id`

to:

`customers.id`

Do not choose cascading deletion for this relationship.

Save the table.

The customer snapshot fields intentionally remain in the invoices table.

They preserve what appeared on the invoice when it was saved.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson is completed inside Supabase.

SAVE YOUR FILES

No project files are created during this lesson.

TEST YOUR WORK

Open the invoices table.

Confirm that:

- ☒ `id` is the Primary Key.
- ☒ `user_id` and `customer_id` are required.
- ☒ `invoice_number` exists.
- ☒ The customer snapshot fields exist.
- ☒ `issue_date` and `due_date` exist.
- ☒ `status` defaults to draft.
- ☒ All discount, tax and total fields exist.

- ✓ created_at and updated_at exist.
- ✓ customer_id relates to customers.id.
- ✓ Row Level Security remains enabled.

CHECKPOINT

Before moving on, confirm that:

- ✓ The invoices table exists.
- ✓ Every required column exists.
- ✓ Every optional column exists.
- ✓ The Primary Key is correct.
- ✓ Row Level Security remains enabled.

COMMON BEGINNER MISTAKES

- One common mistake is making the email field required.
- In many real businesses, an invoice may not have an email address available.
- Phone remains the primary contact method in this workbook.
- Another common mistake is forgetting the updated_at column.
- Although it is not immediately required, it becomes useful later when records are edited and sorted.

BEHIND THE SCENES

- In this table, every row represents one complete invoice.
- As the application grows, the software will retrieve, display and manage these invoices in different ways.
- The structure you create today will support every feature built later.

THINK LIKE A SOFTWARE DESIGNER

- A well-designed table should continue serving the application even after many new features are added.

Good database design anticipates future growth instead of solving only today's problem.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a richer database table
- distinguish required and optional fields
- prepare an invoice database for future features
- keep Row Level Security enabled

In the next lesson, you will learn how to use database constraints to improve data quality before creating the Row Level Security policies.

CHAPTER 4**BUILDING THE INVOICE DATABASE****LESSON 4**

CREATING THE INVOICE ITEMS TABLE

Estimated Time

25 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create the table that stores invoice items.

Each row will represent one line on an invoice.

Several item rows can belong to the same invoice.

WHY THIS MATTERS

Imagine one invoice contains:

Sent

Another contains:

sent

Another contains:

Company

Another contains:

Corporate

Although they all mean almost the same thing, the software now has four different values for one idea.

That makes searching, filtering and reporting much more difficult.

Database constraints reduce this problem by limiting what can be stored.

BEFORE YOU CONTINUE

Confirm that:

- ☒ The invoices table exists.
- ☒ Every required column exists.
- ☒ Every optional column exists.
- ☒ Row Level Security remains enabled.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Create a new table named:

`invoice_items`

Leave Row Level Security enabled.

Create these columns:

- `id` – `uuid` – Primary Key – default value `gen_random_uuid()`
- `invoice_id` – `uuid` – required
- `description` – `text` – required
- `quantity` – `numeric` – required
- `unit_price` – `numeric` – required
- `line_total` – `numeric` – required
- `position` – `integer` – required – default value `0`
- `created_at` – `timestamp with time zone` – required – default value `now()`

Create a relationship from:

`invoice_items.invoice_id`

to:

`invoices.id`

Choose:

Delete related records when the invoice is deleted.

This is also called:

ON DELETE CASCADE

Save the table.

The invoices table stores the complete document.

The invoice_items table stores every line on that document.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No project files are created or updated during this lesson.

TEST YOUR WORK

Open the invoice_items table.

Confirm that:

- ☒ id is the Primary Key.
- ☒ invoice_id is required.
- ☒ description, quantity, unit_price and line_total are required.
- ☒ position exists.
- ☒ invoice_id relates to invoices.id.
- ☒ Deleting an invoice will delete its related items.
- ☒ Row Level Security remains enabled.

CHECKPOINT

Before moving on, confirm that you understand:

- ☒ Why consistent values matter.
- ☒ Which fields are required.

- ☑ Which fields are optional.

COMMON BEGINNER MISTAKES

A common mistake is allowing several different words to represent the same idea.

For example:

Sent

sent

BUSINESS

Company

Corporate

Keeping one standard value makes future searches, filters and reports much easier to build.

BEHIND THE SCENES

Your application will later display filters based on status.

If every record stores different values, those filters become unreliable.

Using consistent values from the beginning prevents unnecessary problems later.

THINK LIKE A SOFTWARE DESIGNER

Good software should encourage consistent information.

Small decisions made early in a project often make future development much simpler.

WHAT YOU LEARNED

In this lesson you learned how to:

- improve data quality
- use consistent values
- prepare the database for reliable searching and filtering

CHAPTER 4

BUILDING THE INVOICE DATABASE

LESSON 5

ADDING DATABASE CONSTRAINTS

Estimated Time

30 minutes

WHAT YOU ARE BUILDING

In this lesson, you will add database rules that reject invalid invoice information.

These rules protect the database even if incorrect information reaches Supabase from outside the normal form.

WHY THIS MATTERS

Browser validation helps the user, but it is not the final security boundary.

A user can change browser code or send a request in another way.

The database must still refuse impossible dates, quantities and totals.

BEFORE YOU CONTINUE

Confirm that:

- ☒ customers exists.
- ☒ invoices exists.
- ☒ invoice_items exists.
- ☒ The table relationships are correct.

BUILD PROMPT

PROMPT STARTS HERE

I have a Supabase project containing:

- customers
- invoices
- invoice_items

Return one complete SQL file named:

invoice-database-constraints.sql

Return the complete file only.

Do not return snippets.

Do not return explanations.

The SQL file must add clearly named constraints that enforce:

CUSTOMERS

- customer_name cannot be empty after spaces are removed

INVOICES

- invoice_number is unique for each user
- status must be draft, sent, paid or overdue
- discount_type must be none, percentage or fixed
- due_date cannot be earlier than issue_date
- discount_value cannot be negative
- subtotal cannot be negative
- discount_amount cannot be negative
- discount_amount cannot be greater than subtotal
- tax_rate cannot be negative
- tax_amount cannot be negative
- total cannot be negative

INVOICE_ITEMS

- description cannot be empty after spaces are removed
 - quantity must be greater than zero
 - unit_price cannot be negative
 - line_total cannot be negative
- Use ALTER TABLE statements.

Give every constraint a clear name.

Do not delete any table.

Do not remove any column.

Do not disable Row Level Security.

The file should be safe to run once on the tables described above.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return one complete file:

invoice-database-constraints.sql

The file should contain all required constraints and no destructive table commands.

SAVE YOUR FILES

Open Notepad.

Paste the complete SQL returned by ChatGPT.

Save the file as:

invoice-database-constraints.sql

Use:

Save as type: All Files

Confirm that the file does not end with:

.txt

TEST YOUR WORK

Open the complete SQL file in Notepad.

Confirm that:

☒ Every constraint has a clear name.

☒ No table is deleted.

☒ No column is removed.

☒ Row Level Security is not disabled.

Copy the complete SQL.

Open:

Supabase

SQL Editor

Create a new query.

Paste the complete SQL.

Run the query once.

Confirm that it completes successfully.

Open the table information for customers, invoices and invoice_items.

Confirm that the new constraints appear.

CHECKPOINT

Before moving on, confirm that the database now rejects:

☒ Empty required names and descriptions

☒ Invalid invoice statuses

☒ Invalid discount types

☒ Due dates before issue dates

☒ Negative totals

☒ Zero or negative quantities

☒ Duplicate invoice numbers for the same user

COMMON BEGINNER MISTAKES

A common mistake is relying only on the HTML form.

The form is useful, but the database must enforce important rules too.

Another mistake is running the same constraint file repeatedly.

Run it once and confirm that the constraints exist before continuing.

BEHIND THE SCENES

- Validation in the browser improves the experience.
- Validation inside the save function protects the complete saving process.
- Constraints protect the stored data at the database level.
- Professional software uses these layers together.

THINK LIKE A SOFTWARE DESIGNER

- Important business rules should be enforced as close to the data as possible.
- This keeps the database trustworthy even when the application changes later.

WHAT YOU LEARNED

- In this lesson you learned how to:
- add database constraints
 - protect invoice data quality
 - reject impossible values
 - combine browser and database validation

CHAPTER 4

BUILDING THE INVOICE DATABASE

LESSON 6

ENABLING ROW LEVEL SECURITY

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will confirm that Row Level Security is enabled for the invoices table.

This is one of the most important security features in the entire application.

WHY THIS MATTERS

Every authenticated user will store invoices inside the same database table.

Without Row Level Security, users could potentially access invoices that belong to someone else.

Row Level Security prevents this.

BEFORE YOU CONTINUE

Confirm that:

- ☒ The invoices table exists.
- ☒ Authentication is working.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Open the Table Editor.

Select:

customers

Confirm that Row Level Security is enabled.

Now select:

invoices

Confirm that Row Level Security is enabled.

Now select:

invoice_items

Confirm that Row Level Security is enabled.

Do not disable Row Level Security on any table.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No project files are created or updated.

TEST YOUR WORK

Confirm that Row Level Security is enabled on:

☒ customers

☒ invoices

☒ invoice_items

CHECKPOINT

Before moving on, confirm that:

☒ Row Level Security remains enabled.

COMMON BEGINNER MISTAKES

Some beginners disable Row Level Security while trying to solve another problem.

Do not do this.

Throughout this workbook, Row Level Security should remain enabled.

BEHIND THE SCENES

Authentication identifies the current user.

Row Level Security decides what that user is allowed to access.

Both are required for secure software.

THINK LIKE A SOFTWARE DESIGNER

Security should be built into the application from the beginning.

It should never become an afterthought.

WHAT YOU LEARNED

In this lesson you learned how to:

- confirm Row Level Security
- understand its relationship with authentication
- prepare the database for secure invoices

CHAPTER 4

BUILDING THE INVOICE DATABASE

LESSON 7

CREATING THE SELECT POLICIES

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create the SELECT policy for the invoices table.

This policy controls which invoices a user is allowed to view.

WHY THIS MATTERS

Without a SELECT policy, users cannot safely retrieve invoices.

The policy ensures that every user sees only their own invoices.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Row Level Security is enabled.
- ☒ The invoices table exists.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Create the SELECT policies directly inside Supabase.

Open:

Authentication

Then:

Policies

CUSTOMERS

Create a SELECT policy for:

customers

Use:

`user_id = auth.uid()`

in:

USING

INVOICES

Create the matching SELECT policy for:

invoices

Use:

`user_id = auth.uid()`

in USING.

INVOICE_ITEMS

Create the SELECT policy for:

invoice_items

The invoice_items table does not store a separate user_id.

Use an EXISTS check in USING. Confirm that invoice_items.invoice_id matches invoices.id and invoices.user_id matches auth.uid().

Review the three policies carefully.

Save every policy.

If you ask AI to prepare SQL, ask for one complete SQL file.

Do not request a snippet.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No project files are created or updated.

TEST YOUR WORK

Return to the Policies page.

Confirm that:

- ☒ customers has a SELECT policy.
- ☒ invoices has a SELECT policy.
- ☒ invoice_items has a SELECT policy.

Confirm that the customers and invoices policies compare:

user_id

with:

auth.uid()

Confirm that the invoice_items policy checks ownership through the parent invoice.

Do not continue while any policy is missing.

CHECKPOINT

Before moving on, confirm that:

- ☒ The SELECT policy exists.
- ☒ The policy has been saved.
- ☒ The policy compares:

user_id

with

`auth.uid()`

COMMON BEGINNER MISTAKES

A common mistake is comparing the wrong field.

Always compare:

`user_id`

with:

`auth.uid()`

No other comparison should be used.

BEHIND THE SCENES

Whenever your application requests invoices, Supabase checks this policy before returning any data.

Only rows belonging to the authenticated user will be returned.

THINK LIKE A SOFTWARE DESIGNER

Security should happen automatically.

Your application should not have to remember to protect every request.

The database should enforce those rules itself.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a SELECT policy
- protect invoices
- allow users to view only their own information

CHAPTER 4

BUILDING THE INVOICE DATABASE

LESSON 8

CREATING THE INSERT POLICIES

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create the INSERT policy.

This policy controls which new invoices may be saved.

WHY THIS MATTERS

Every new invoice should belong to the authenticated user who created it.

The INSERT policy ensures that only authorised records can be stored.

BEFORE YOU CONTINUE

Confirm that:

- ☒ The SELECT policy exists.
- ☒ Row Level Security remains enabled.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Create the INSERT policies directly inside Supabase.

Open:

Authentication

Then:

Policies

CUSTOMERS

Create a INSERT policy for:

customers

Use:

```
user_id = auth.uid()
```

in:

WITH CHECK

INVOICES

Create the matching INSERT policy for:

invoices

In WITH CHECK, require both:

- `invoices.user_id` matches `auth.uid()`
- An EXISTS check confirms that `invoices.customer_id` belongs to a customers row whose `user_id` matches `auth.uid()`

This prevents a forged request from attaching an invoice to another user's customer.

INVOICE_ITEMS

Create the INSERT policy for:

invoice_items

The `invoice_items` table does not store a separate `user_id`.

Use an EXISTS check in WITH CHECK. Confirm that the parent invoice belongs to `auth.uid()`.

Review the three policies carefully.

Save every policy.

If you ask AI to prepare SQL, ask for one complete SQL file.

Do not request a snippet.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No project files are created or updated.

TEST YOUR WORK

Return to the Policies page.

Confirm that:

- ☒ customers has a INSERT policy.
- ☒ invoices has a INSERT policy.
- ☒ invoice_items has a INSERT policy.

Confirm that the customers and invoices policies compare:

user_id

with:

auth.uid()

Confirm that the invoice_items policy checks ownership through the parent invoice.

Confirm that the invoices policy also checks that the selected customer belongs to auth.uid().

Do not continue while any policy is missing.

CHECKPOINT

Before moving on, confirm that:

- ☒ The INSERT policy exists.
- ☒ The policy has been saved successfully.

COMMON BEGINNER MISTAKES

Some beginners believe that authentication alone allows records to be inserted.

Authentication identifies the user.

The INSERT policy gives permission to save the record.

Both are required.

BEHIND THE SCENES

Whenever your application creates a new invoice, Supabase checks this policy before saving the record.

If the invoice belongs to the authenticated user, the record is paid.

Otherwise, it is rejected.

THINK LIKE A SOFTWARE DESIGNER

Every database action should have its own security rule.

Viewing data and saving data are two different permissions.

WHAT YOU LEARNED

In this lesson you learned how to:

- create an INSERT policy
- protect new invoices
- understand how authentication and Row Level Security work together

In the next lessons, you will create the UPDATE and DELETE policies before carrying out a complete security audit of the invoice database.

CHAPTER 4

BUILDING THE INVOICE DATABASE

LESSON 9

CREATING THE UPDATE POLICIES

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create the UPDATE policy for the invoices table.

This policy controls which invoices users are allowed to modify.

Once this lesson is complete, authenticated users will be able to update only the invoices they own.

WHY THIS MATTERS

Invoice information changes over time.

An invoice may:

- Change their phone number.
- Change their email address.
- Move to another city.
- Start working for a different company.
- Receive additional notes.

Your application should allow those changes while ensuring users cannot modify another person's invoices.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Row Level Security is enabled.
- ☒ The SELECT policy exists.

✓ The INSERT policy exists.

✓ The invoices table exists.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Create the UPDATE policies directly inside Supabase.

Open:

Authentication

Then:

Policies

CUSTOMERS

Create a UPDATE policy for:

customers

Use:

`user_id = auth.uid()`

in:

USING and WITH CHECK

INVOICES

Create the matching UPDATE policy for:

invoices

In USING, require:

`user_id = auth.uid()`

In WITH CHECK, require both:

- `invoices.user_id` matches `auth.uid()`
- An EXISTS check confirms that `invoices.customer_id` belongs to a customers row whose `user_id` matches `auth.uid()`

This protects the existing invoice and any newly selected customer.

INVOICE_ITEMS

Create the UPDATE policy for:

invoice_items

The invoice_items table does not store a separate user_id.

Use the same parent-invoice ownership EXISTS check in both USING and WITH CHECK.

Review the three policies carefully.

Save every policy.

If you ask AI to prepare SQL, ask for one complete SQL file.

Do not request a snippet.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson is completed inside Supabase.

SAVE YOUR FILES

No project files are created or updated during this lesson.

TEST YOUR WORK

Return to the Policies page.

Confirm that:

- ☒ customers has a UPDATE policy.
- ☒ invoices has a UPDATE policy.
- ☒ invoice_items has a UPDATE policy.

Confirm that the customers and invoices policies compare:

user_id

with:

auth.uid()

Confirm that the `invoice_items` policy checks ownership through the parent invoice.

Confirm that the `invoices` policy also checks that the selected customer belongs to `auth.uid()`.

Do not continue while any policy is missing.

CHECKPOINT

Before moving on, confirm that:

- ☒ The `UPDATE` policy exists.
- ☒ The `UPDATE` policy has been saved.
- ☒ The policy compares:

`user_id`

with

`auth.uid()`

COMMON BEGINNER MISTAKES

Some beginners believe that identifying an invoice by its ID is enough when updating it.

It is not.

Later in this workbook, your JavaScript will update invoices by checking both:

- The invoice ID
- The authenticated user's ID

The database will then apply the `UPDATE` policy as an additional layer of protection.

BEHIND THE SCENES

Every time someone edits an invoice, Supabase checks the `UPDATE` policy before saving the changes.

If the record belongs to the authenticated user, the update succeeds.

If it belongs to another user, the request is rejected automatically.

THINK LIKE A SOFTWARE DESIGNER

Reading information and changing information are different permissions.

Professional software treats them separately.

Just because someone can see an invoice does not automatically mean they should be allowed to change it.

WHAT YOU LEARNED

In this lesson you learned how to:

- create an UPDATE policy
- protect invoices during editing
- understand why updates require their own permission

CHAPTER 4

BUILDING THE INVOICE DATABASE

LESSON 10

CREATING THE DELETE POLICIES

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create the final Row Level Security policy for your invoices table.

This policy controls which invoices users are allowed to remove.

After completing this lesson, every important database action will have its own security policy.

WHY THIS MATTERS

Deleting an invoice permanently removes information from the database.

Your application must ensure that users can delete only the invoices they own.

BEFORE YOU CONTINUE

Confirm that:

- ☒ The SELECT policy exists.
- ☒ The INSERT policy exists.
- ☒ The UPDATE policy exists.
- ☒ Row Level Security remains enabled.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Create the DELETE policies directly inside Supabase.

Open:

Authentication

Then:

Policies

CUSTOMERS

Create a DELETE policy for:

customers

Use:

`user_id = auth.uid()`

in:

USING

INVOICES

Create the matching DELETE policy for:

invoices

Use:

`user_id = auth.uid()`

in USING.

INVOICE_ITEMS

Create the DELETE policy for:

invoice_items

The `invoice_items` table does not store a separate `user_id`.

Use an EXISTS check in USING. Confirm that the parent invoice belongs to `auth.uid()`.

Review the three policies carefully.

Save every policy.

If you ask AI to prepare SQL, ask for one complete SQL file.

Do not request a snippet.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No project files are created or updated during this lesson.

TEST YOUR WORK

Return to the Policies page.

Confirm that:

- ☒ customers has a DELETE policy.
- ☒ invoices has a DELETE policy.
- ☒ invoice_items has a DELETE policy.

Confirm that the customers and invoices policies compare:

user_id

with:

auth.uid()

Confirm that the invoice_items policy checks ownership through the parent invoice.

Do not continue while any policy is missing.

CHECKPOINT

Before moving on, confirm that:

- ☒ The DELETE policy exists.
- ☒ The DELETE policy has been saved successfully.
- ☒ The policy compares:

user_id

with

auth.uid()

COMMON BEGINNER MISTAKES

Some beginners believe that because a user created an invoice, they automatically have permission to delete it.

That permission comes from the DELETE policy.

Without it, the database rejects the request.

BEHIND THE SCENES

Whenever your application attempts to delete an invoice, Supabase checks the DELETE policy first.

If the invoice belongs to the authenticated user, the record is removed.

If not, the request is rejected automatically.

THINK LIKE A SOFTWARE DESIGNER

Deleting information should always receive special attention.

Professional software protects permanent actions with multiple layers of security instead of assuming every request is valid.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a DELETE policy
- protect invoices from unauthorised deletion
- complete the four essential Row Level Security policies

CHAPTER 4

BUILDING THE INVOICE DATABASE

LESSON 11

TESTING THE INVOICE DATABASE

Estimated Time

30 minutes

WHAT YOU ARE BUILDING

In this lesson, you will verify that your invoice database has been built correctly.

You are not creating new tables or policies.

Instead, you will review everything completed so far before invoice management begins.

WHY THIS MATTERS

The invoice database is the foundation of every feature that follows.

If there is a mistake in the database structure or security policies, adding, editing and deleting invoices may not work correctly.

Professional developers review their database carefully before writing application features.

BEFORE YOU CONTINUE

Confirm that:

- ☒ The invoices table exists.
- ☒ Row Level Security remains enabled.
- ☒ All four policies have been created.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, complete the following review.

Open:

Table Editor

Open:

invoices

Review every column.

Next, open:

Authentication

Policies

Review every policy.

Confirm that:

SELECT

uses:

user_id = auth.uid()

INSERT

uses:

user_id = auth.uid()

UPDATE

uses:

user_id = auth.uid()

DELETE

uses:

user_id = auth.uid()

Finally, confirm that Row Level Security is still enabled.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No project files are created or updated.

TEST YOUR WORK

Complete this test with two different accounts on the deployed HTTPS website.

ACCOUNT A

Sign in as Account A.

Create one customer and one invoice when the application features are available.

Confirm that the customer and invoice belong to Account A.

Sign out.

ACCOUNT B

Sign in as Account B.

Confirm that Account B cannot view, update or delete Account A's customer, invoice or invoice items.

Create a separate customer and invoice for Account B.

Confirm that Account B can access only its own information.

SUPABASE REVIEW

Confirm that all four operations have policies on:

☒ customers

☒ invoices

☒ invoice_items

Do not continue if either account can access the other account's data.

CHECKPOINT

Before moving on, confirm that:

- ✓ The invoices table is complete.
- ✓ Every required column exists.
- ✓ Every policy exists.
- ✓ Row Level Security remains enabled.
- ✓ No policy has been removed accidentally.

COMMON BEGINNER MISTAKES

Some students accidentally delete a policy while experimenting inside Supabase.

Before moving to the next chapter, confirm that all four policies are still present.

Another common mistake is forgetting that every policy should use the same ownership check:

```
user_id = auth.uid()
```

Changing one policy while leaving the others unchanged can produce confusing behaviour later.

BEHIND THE SCENES

Your invoice database is now fully prepared.

The next chapter will not change the database.

Instead, it will begin using this database to store and retrieve real invoice information.

Because the database already protects every row, your application can focus on providing a good user experience while Supabase continues enforcing security behind the scenes.

THINK LIKE A SOFTWARE DESIGNER

A secure application begins with a secure database.

When the foundation is built correctly, every feature added later becomes easier to develop and easier to trust.

Professional developers invest time in getting the database right before writing business logic.

WHAT YOU LEARNED

In this lesson you learned how to:

- review an invoice database
- verify Row Level Security
- verify every database policy
- confirm that the application is ready for invoice management

CHAPTER SUMMARY

Congratulations.

Your Professional Invoice Generator now has a complete three-table database.

Customer profiles can be reused, invoices preserve a customer snapshot, and every invoice can contain several related items.

Database constraints protect important business rules.

Row Level Security and ownership policies protect customers, invoices and invoice items for every operation.

CHAPTER MILESTONE

By the end of Chapter 4, you have successfully built:

- ✓ customers table
- ✓ invoices table
- ✓ invoice_items table
- ✓ Correct relationships
- ✓ Customer snapshot design
- ✓ Database constraints
- ✓ Row Level Security
- ✓ SELECT policies
- ✓ INSERT policies
- ✓ UPDATE policies
- ✓ DELETE policies
- ✓ Two-account security test

TRANSITION TO CHAPTER 5

The secure database foundation is complete.

In Chapter 5, you will build the working invoice workspace.

Users will save and select customers, prepare invoices containing several items, calculate discounts and tax, save the complete invoice securely, and view live dashboard statistics.

CHAPTER 5

BUILDING THE INVOICE DASHBOARD

CHAPTER INTRODUCTION

Your Professional Invoice Generator now has secure database tables and policies.

In this chapter, you will build the main working area.

The user will save customer profiles, select one customer, add several invoice items, calculate totals and save the complete invoice.

The browser will provide immediate calculations.

Supabase will independently validate the customer, items and trusted totals before saving.

The dashboard will also display useful invoice statistics.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- Responsive invoice workspace
- Customer-saving form

- Select Customer field and preview
- Multiple invoice items
- Add Item and Remove Item controls
- Automatic line totals
- Subtotal, discount, tax and final total
- Secure transactional invoice saving
- Customer snapshot saving
- Total, draft, sent and paid statistics

LESSON 1

DESIGNING THE INVOICE WORKSPACE

Estimated Time

50 minutes

WHAT YOU ARE BUILDING

In this lesson, you will design the main working area of your Professional Invoice Generator.

This dashboard is where users will spend most of their time after signing in.

Rather than displaying a blank page, the dashboard will present useful business information alongside an organised invoice management area.

The invoice form and business statistics you build later in this chapter will fit naturally into this layout.

WHY THIS MATTERS

Business software is used repeatedly throughout the working day.

A good dashboard helps users understand where they are, what they can do next and how the business is performing.

A poorly organised dashboard slows people down.

Before adding invoice management features, it is worth investing time in creating a clean, professional workspace.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Registration works correctly.
- ☒ Email verification works.
- ☒ Login works.
- ☒ Dashboard authentication protection works.
- ☒ Logout works.

Only authenticated users should be able to reach the dashboard.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Professional Invoice Generator.

Authentication and dashboard protection work.

Everything already built must continue working.

Return complete updated files only.

Do not return snippets.

Update only:

- dashboard.html
- dashboard.js
- auth.css

GENERAL GOAL

Build the complete visual structure for the invoice workspace.

DASHBOARD.HTML

Keep authentication protection and logout.

Add:

- Dashboard header
- Signed-in user message
- Four invoice statistic cards
- Customer area
- Select Customer field
- Create Invoice section
- Business Details section
- Invoice Details section
- Invoice Items section

- Totals section
- Save Invoice button
- Status message area
- Invoice History section

The detailed customer and invoice behaviour will be completed in later lessons.

DASHBOARD.JS

Keep authentication, logout and the signed-in user message working.

Do not save customers or invoices yet.

AUTH.CSS

Keep every existing style.

Create a clean, responsive workspace.

IMPORTANT

Do not remove any authentication feature.

Do not create another Supabase client.

Return complete updated files only.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return complete updated versions of:

- dashboard.html
- dashboard.js
- auth.css

The dashboard should now provide a professional workspace ready for invoice management.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Open:

dashboard.html

in Notepad.

Replace its complete contents.

Save the file.

Repeat the same process for:

dashboard.js

and

auth.css.

TEST YOUR WORK

Sign in and open the dashboard.

Confirm that:

☒ The dashboard header and signed-in user message appear.

☒ Four invoice statistic cards appear.

☒ The customer area appears.

☒ Select Customer appears.

☒ The Create Invoice structure appears.

☒ Business, invoice items, totals and history sections appear.

✓ Save Invoice and Logout appear.

Resize the browser.

Confirm that the workspace remains readable without unnecessary horizontal scrolling.

CHECKPOINT

Before moving on, confirm that:

✓ The dashboard layout is complete.

✓ All invoice fields are visible.

✓ The dashboard remains protected.

✓ Existing authentication continues working.

✓ Every future section has a clear placeholder.

COMMON BEGINNER MISTAKES

Some beginners immediately begin connecting forms to the database before finishing the dashboard layout.

Doing so makes it much harder to recognise whether a problem comes from the design or the JavaScript.

Complete the structure first.

Then add behaviour.

BEHIND THE SCENES

Professional software is usually developed in layers.

The first layer provides the visual structure.

The next layer adds behaviour.

The final layer connects the interface to the database.

Separating those responsibilities makes the software easier to build and maintain.

THINK LIKE A SOFTWARE DESIGNER

A good dashboard should answer three questions immediately:

Where am I?

What can I do here?

What information is most important?

Design every dashboard with those questions in mind.

CODE-READING QUESTION

Open `dashboard.js`. Find one function that supports designing the invoice workspace. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- build a professional business dashboard
- organise information into logical sections
- prepare a dashboard for future functionality
- design software before connecting it to a database

CHAPTER 5

BUILDING THE INVOICE WORKSPACE

LESSON 2

SAVING AND SELECTING CUSTOMERS

Estimated Time

50 minutes

WHAT YOU ARE BUILDING

In this lesson, you will add a simple customer area to the dashboard.

The user will be able to save a customer and select that customer while preparing an invoice.

WHY THIS MATTERS

Typing the same customer details into every invoice wastes time.

A saved customer list makes invoice creation faster and keeps customer information consistent.

BEFORE YOU CONTINUE

Confirm that:

- ☒ The customers table exists.
- ☒ Customer Row Level Security policies exist.
- ☒ The dashboard is protected.
- ☒ Your latest project folder is backed up.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Professional Invoice Generator.

Registration, login, logout and dashboard protection work.

The customers, invoices and invoice_items tables already exist.

Row Level Security and authenticated ownership policies are enabled.

Everything already built must continue working.

Continue using the shared supabaseClient from supabase-config.js.

Return complete updated files only.

Do not return snippets or partial code.

Update only:

- dashboard.html
- dashboard.js
- auth.css

GENERAL GOAL

Build one complete customer-saving and customer-selection capability.

DASHBOARD.HTML

Keep every existing dashboard section.

Add a customer area containing:

- Customer name
- Company name
- Email
- Phone
- Billing address
- Save Customer button
- Friendly status message

Add a Select Customer field to the invoice form.

Add a small customer preview below the selection field.

The preview should display the selected customer's saved information.

DASHBOARD.JS

Keep authentication, logout and dashboard protection working.

After authentication succeeds:

- Load only customers available to the signed-in user
- Order customers by customer_name
- Populate the Select Customer field
- Show a friendly empty state when no customer exists

When Save Customer is selected:

- Validate customer_name
- Trim all text values
- Convert email to lowercase
- Confirm that the user is authenticated
- Insert the customer using the current user's ID
- Disable the button while saving
- Display friendly success and error messages
- Clear the customer form only after a successful save
- Reload the customer selection list

When a customer is selected:

- Store the selected customer ID
- Display the customer preview
- Do not allow a customer belonging to another user to be selected

Do not create another Supabase client.

Do not use the browser's local storage as the customer database.

AUTH.CSS

Keep every existing style.

Style the customer form, selection field, preview, empty state and messages.

Keep the complete area readable on small screens.

IMPORTANT

Return all three complete updated files.

Do not return snippets.

Do not remove any working feature.

PROMPT ENDS HERE**WHAT AI SHOULD RETURN**

ChatGPT should return three complete updated files:

- dashboard.html
- dashboard.js
- auth.css

The dashboard should now save customers securely and allow the signed-in user to select a saved customer.

SAVE YOUR FILES**BACK UP BEFORE REPLACING FILES**

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Replace the complete contents of:

dashboard.html

dashboard.js

auth.css

Use the backup instructions before replacing the files.

Save every file using its exact filename.

TEST YOUR WORK

Open the deployed HTTPS website.

Sign in.

Open the dashboard.

Confirm that the customer form and Select Customer field appear.

Try to save the form without a customer name.

A friendly validation message should appear.

Now save one customer.

Confirm that:

☒ The Save Customer button is disabled while saving.

☒ A success message appears.

☒ The customer form clears.

☒ The new customer appears in Select Customer.

Select the customer.

Confirm that the saved customer details appear in the preview.

Refresh the page.

Confirm that the customer remains available.

Open the customers table in Supabase.

Confirm that the saved row contains the authenticated user's ID.

CHECKPOINT

Before moving on, confirm that:

☒ Customers save successfully.

☒ The selection list loads successfully.

☒ The customer preview works.

- ✓ Friendly loading, empty and error states appear.

COMMON BEGINNER MISTAKES

A common mistake is loading customers before authentication has finished.

Another mistake is saving a customer without the current user's ID.

Do not remove the ownership value.

The database policies and the application query must work together.

BEHIND THE SCENES

The selection field stores a customer ID, not only a customer name.

That ID creates a reliable relationship between the customer and a future invoice.

THINK LIKE A SOFTWARE DESIGNER

Reusable information should be saved once and selected when needed.

This reduces repeated work and creates a more consistent business process.

CODE-READING QUESTION

Open dashboard.js. Find one function that supports saving and selecting customers. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- save customer profiles
- load user-owned customers
- build a customer selection field
- connect a selected customer to a future invoice

CHAPTER 5

BUILDING THE INVOICE DASHBOARD

LESSON 3

BUILDING THE INVOICE FORM

Estimated Time

70 minutes

WHAT YOU ARE BUILDING

In the previous lesson, you designed the Invoice Dashboard and prepared space for the invoice form.

In this lesson, you will build a professional invoice entry form that businesses can use to collect invoice information consistently.

The form will validate user input, clean invoice information automatically, and prepare every invoice for secure storage.

The information will not be saved into Supabase yet.

Saving invoices will be completed in the next lesson.

WHY THIS MATTERS

Good invoices begin with good data entry.

If users can enter incomplete or inconsistent information, the invoice database quickly becomes difficult to search and maintain.

A professional invoice form helps prevent those problems before information reaches the database.

BEFORE YOU CONTINUE

Confirm that:

- ☒ The Invoice Dashboard appears correctly.
- ☒ All dashboard sections are visible.
- ☒ The Invoice History placeholder appears.

✓ Dashboard authentication still works.

✓ Logout still works.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Professional Invoice Generator.

Saving and selecting customers already works.

Everything already built must continue working.

Return complete updated files only.

Do not return snippets.

Update only:

- dashboard.html
- dashboard.js
- auth.css

GENERAL GOAL

Build the complete invoice form and automatic browser calculations.

DASHBOARD.HTML

Keep the customer form and Select Customer field.

The invoice form must include:

- Selected customer preview
- Message explaining that the invoice number is generated when saved
- Business name
- Business email
- Business phone
- Business address
- Issue date
- Due date

- Status with draft, sent, paid and overdue options

- Currency

- Notes

Add an invoice items section.

Each item row must include:

- Description

- Quantity

- Unit price

- Line total

Add:

- Add Item button

- Remove Item button on every item after the first

Add a totals section containing:

- Subtotal

- Discount type

- Discount value

- Discount amount

- Tax rate

- Tax amount

- Final total

DASHBOARD.JS

Keep customer saving, customer selection, authentication and logout working.

Require one selected customer.

Allow users to add and remove item rows.

Always keep at least one item row.

Recalculate line totals when quantity or unit price changes.

Recalculate the complete invoice when an item, discount or tax value changes.

Support no discount, percentage discount and fixed discount.

Apply tax after the discount.

Reject negative values and zero quantity.

Do not allow a discount larger than the subtotal.

Display money values to two decimal places.

Do not save an invoice in this lesson.

AUTH.CSS

Keep every existing style.

Style the form, item rows, totals and buttons.

Keep the form readable on small screens.

IMPORTANT

Return all three complete files.

Do not remove anything that already works.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return complete updated versions of:

- dashboard.html
- dashboard.js
- auth.css

The invoice form should now support customer selection, item rows and complete live calculations.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Open:

dashboard.html

in Notepad.

Replace its complete contents.

Save the file.

Repeat the same process for:

dashboard.js

and

auth.css.

TEST YOUR WORK

Open the deployed dashboard.

Select a saved customer.

Add three invoice items with different quantities and prices.

Confirm that:

☒ Every line total changes correctly.

☒ The subtotal equals all line totals.

☒ Percentage discount works.

☒ Fixed discount works.

☒ Tax is applied after the discount.

☒ The final total is correct.

☒ Removing an item updates every total.

☒ At least one item always remains.

Try a negative price and zero quantity.

Friendly validation should prevent both values.

Do not continue until every calculation is correct.

CHECKPOINT

Before moving on, confirm that:

- ✓ A saved customer can be selected.
- ✓ At least one item is always required.
- ✓ Quantity and price validation work.
- ✓ Discount and tax calculations work.
- ✓ No invoice has been saved in this lesson.

COMMON BEGINNER MISTAKES

A common mistake is treating the totals displayed by the browser as final trusted values.

The browser calculations help the user while the invoice is being prepared.

Supabase will calculate the trusted values again when the invoice is saved.

BEHIND THE SCENES

The form gives immediate feedback whenever an item, discount or tax value changes.

This makes the application feel responsive without weakening the database validation used during saving.

THINK LIKE A SOFTWARE DESIGNER

Every invoice added today may still exist in the business database years from now.

Investing in clean data entry now saves countless hours of correction later.

CODE-READING QUESTION

Open dashboard.js. Find one function that supports building the invoice form. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- require a selected customer
- create and remove invoice item rows
- calculate line totals and subtotal
- apply discounts and tax
- prepare a complete invoice for secure saving

CHAPTER 5

BUILDING THE INVOICE DASHBOARD

LESSON 4

CALCULATING AND SAVING INVOICES SECURELY

Estimated Time

90 minutes

WHAT YOU ARE BUILDING

In this lesson, you will complete secure invoice saving.

One database function will verify the selected customer, recalculate every trusted total, save the invoice, and save all its items as one complete action.

If any part fails, no partial invoice will remain.

WHY THIS MATTERS

Creating invoices is one of the most important responsibilities of this software.

The workflow must protect data quality, maintain invoice privacy, and ensure that every record belongs only to the authenticated user.

Once this lesson is complete, businesses will be able to trust the invoice information stored inside the application.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Invoice form validation works.
- ☒ Invoice fields and items are validated.
- ☒ The invoices table exists.
- ☒ Row Level Security remains enabled.
- ☒ INSERT policy exists.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Professional Invoice Generator.

Customer selection, invoice items and browser calculations work.

Everything already built must continue working.

Return complete files only.

Do not return snippets.

Create or update only:

- save-invoice-function.sql
- dashboard.html
- dashboard.js
- auth.css

GENERAL GOAL

Save one complete invoice and all its items as one secure database action.

SAVE-INVOICE-FUNCTION.SQL

Create one complete PostgreSQL function named:

save_invoice_with_items

Use the normal authenticated caller and Row Level Security.

The function must:

- Require auth.uid()
- Accept customer_id, invoice details and a JSON array of items
- Confirm that the selected customer belongs to auth.uid()
- Copy the selected customer details into the invoice snapshot fields
- Validate issue date and due date
- Validate at least one item
- Validate every description, quantity and unit price

- Generate an invoice number unique for the signed-in user
 - Recalculate every line total in Supabase
 - Recalculate subtotal, discount, tax and final total in Supabase
 - Never trust totals received from the browser
 - Insert the invoice
 - Insert every invoice item
 - Complete all inserts as one transaction
 - Leave no partly saved invoice when an error occurs
 - Return the new invoice ID and invoice number
- Restrict execution to authenticated users.

DASHBOARD.JS

- Keep every existing capability.
- When Save Invoice is selected:
- Require a selected customer
 - Validate all required invoice fields and items
 - Confirm the user is authenticated
 - Call `save_invoice_with_items`
 - Disable the button while saving
 - Display friendly progress, success and error messages
 - Clear the invoice form only after success
 - Keep saved customers available
 - Refresh invoice history and statistics
- Use `console.error()` for technical details.
- Do not display raw database messages to the learner.

IMPORTANT

- Return all four complete files.
- Do not create another Supabase client.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return complete updated versions of:

- dashboard.html
- dashboard.js
- auth.css

The Professional Invoice Generator should now support the complete invoice creation workflow.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Replace the complete contents of every updated file.

Save each file.

TEST YOUR WORK

Sign in on the deployed website.

Select a saved customer.

Prepare an invoice containing at least two items, a discount and tax.

Save the invoice.

Confirm that:

- ☒ The button is disabled while saving.

☒ A friendly success message appears.

☒ The generated invoice number appears.

☒ The invoice form clears only after success.

Open the invoices table in Supabase.

Confirm that:

☒ One invoice row exists.

☒ `user_id` belongs to the signed-in user.

☒ `customer_id` identifies the selected customer.

☒ Customer snapshot fields contain the selected customer's details.

☒ Trusted subtotal, discount, tax and total values are correct.

Open `invoice_items`.

Confirm that every item belongs to the new invoice.

Try saving without a customer and without an item.

Friendly validation should prevent both attempts.

CHECKPOINT

Before moving on, confirm that:

☒ The selected customer is verified.

☒ Supabase recalculates every trusted total.

☒ The invoice and items save together.

☒ A failed request leaves no partial invoice.

☒ Invoice ownership is protected.

☒ The form resets only after success.

COMMON BEGINNER MISTAKES

A common mistake is inserting the invoice and each item through separate browser requests.

If one request fails, the database may contain an incomplete document.

The database function completes the invoice and item inserts as one transaction.

Another mistake is trusting customer or total values supplied by the browser.

Supabase verifies the customer and recalculates the totals.

BEHIND THE SCENES

The database function is responsible for the final trusted result.

It checks ownership, preserves the customer snapshot, calculates the totals and saves related records together.

THINK LIKE A SOFTWARE DESIGNER

Creating an invoice is more than inserting information into a database.

It is a carefully controlled business process that protects data quality, user privacy and future reporting.

CODE-READING QUESTION

Open dashboard.js. Find one function that supports calculating and saving invoices securely. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

-
- verify customer ownership
 - calculate trusted totals in Supabase
 - save an invoice and its items as one transaction
 - prevent partly saved invoices
 - refresh application data after success

CHAPTER 5

BUILDING THE INVOICE DASHBOARD

LESSON 5

BUILDING INVOICE DASHBOARD STATISTICS

Estimated Time

60 minutes

WHAT YOU ARE BUILDING

Your Professional Invoice Generator can now create invoices securely.

In this lesson, you will transform the dashboard from a simple data entry screen into a useful business dashboard.

The dashboard will automatically calculate and display meaningful invoice statistics every time the page loads and whenever a new invoice is added.

Instead of forcing users to count invoices themselves, the software will do the work automatically.

WHY THIS MATTERS

Dashboard statistics turn saved invoice information into a quick business overview.

The user should see how many invoices exist and how many are currently draft, sent or paid without counting them manually.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Complete invoices save successfully.
- ☒ The invoice form resets after success.
- ☒ Test invoices exist with different statuses.
- ☒ Dashboard authentication still works.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Professional Invoice Generator.

Creating and saving complete invoices works.

Everything already built must continue working.

Return complete updated files only.

Do not return snippets.

Update only:

- dashboard.html
- dashboard.js
- auth.css

GENERAL GOAL

Display live invoice statistics.

Use four cards:

- Total Invoices
- Draft Invoices
- Sent Invoices
- Paid Invoices

Load only invoices available to the signed-in user.

Display 0 when no invoice exists.

Refresh the cards when:

- The dashboard opens
- An invoice is saved
- An invoice is edited
- An invoice is deleted

Handle loading and errors with friendly messages.

Continue relying on Row Level Security.

Return complete files only.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return complete updated versions of:

- dashboard.html
- dashboard.js
- auth.css

The dashboard should automatically display live invoice statistics.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Replace the complete contents of every updated file.

Save each file.

TEST YOUR WORK

Open the dashboard.

Confirm that all four cards display numbers.

Create one draft invoice.

Confirm that Total Invoices and Draft Invoices increase.

Create or update an invoice with sent status.

- Confirm that Sent Invoices displays correctly.
- Mark an invoice as paid when editing is available.
- Confirm that Paid Invoices displays correctly.
- The cards should refresh without manually refreshing the browser.

CHECKPOINT

Before moving on, confirm that:

- ✓ Total, draft, sent and paid counts are correct.
- ✓ Statistics refresh automatically.
- ✓ Invoice creation still works.
- ✓ Only the signed-in user's invoices are counted.

COMMON BEGINNER MISTAKES

- Some beginners manually increase the displayed numbers after saving an invoice.
- Professional software recalculates statistics from the database.
- This guarantees that the dashboard always reflects the actual invoices stored in the system.

BEHIND THE SCENES

- The dashboard does not store separate statistics.
- Instead, it calculates them from the invoices already stored in the database.
- Whenever new information is added, the calculations simply run again using the latest data.
- This keeps the dashboard accurate without storing duplicate information.

THINK LIKE A SOFTWARE DESIGNER

- A dashboard should help users make decisions quickly.
- Whenever information can be calculated automatically from existing data, let the software do the work instead of expecting users to calculate it themselves.

CODE-READING QUESTION

Open dashboard.js. Find one function that supports building invoice dashboard statistics. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- calculate business statistics
- display live dashboard information
- refresh statistics automatically
- improve business visibility through software

CHAPTER SUMMARY

Congratulations.

Your application can now save reusable customers and select them while preparing invoices.

Users can add several items, see immediate calculations and save the complete invoice as one secure database action.

Supabase validates ownership and recalculates trusted totals.

The dashboard statistics refresh when invoice information changes.

CHAPTER MILESTONE

By the end of Chapter 5, you have successfully built:

- ✓ Professional invoice workspace
- ✓ Customer saving and selection
- ✓ Customer preview
- ✓ Multiple invoice items
- ✓ Live calculations
- ✓ Trusted Supabase calculations
- ✓ Secure transactional saving
- ✓ Customer snapshot
- ✓ Automatic form reset
- ✓ Live invoice statistics

TRANSITION TO CHAPTER 6

Your application can now create complete invoices.

In Chapter 6, you will replace the history placeholder with a clear list of saved invoices and build a protected page that displays one complete invoice and all its items.

CHAPTER 6

VIEWING INVOICES

CHAPTER INTRODUCTION

A saved invoice becomes useful when the user can find it and open it easily.

At the moment, your invoices exist inside Supabase.

In this chapter, you will display them inside the application.

The dashboard will show a clear history of saved invoices.

Each invoice card will show the invoice number, customer name, dates, status and final total.

Users will also be able to open one invoice and view every item and calculation.

Loading, empty and error messages will help beginners understand what the application is doing.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- Invoice History
- Invoice cards
- Invoice number
- Customer name
- Issue and due dates
- Status

- Final total
- View Invoice action
- Complete invoice details page
- Loading state
- Empty state
- Error state
- Responsive layouts
- Invoice privacy testing

LESSON 1

BUILDING THE INVOICE HISTORY

Estimated Time

90 minutes

WHAT YOU ARE BUILDING

In this lesson, you will build the complete Invoice History.

The application will retrieve authorised invoices from Supabase, display clear invoice cards, and allow users to open one complete invoice and its items.

Invoice History will also provide friendly loading, empty and error states.

WHY THIS MATTERS

Businesses rarely interact directly with database tables.

Instead, they work with organised invoice lists that present information clearly and make common tasks quick and convenient.

A well-designed invoice history allows users to recognise invoices immediately and perform common actions with very little effort.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Invoices save successfully.
- ☒ Dashboard statistics work.
- ☒ Several invoices exist.
- ☒ The Invoice History placeholder appears.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Professional Invoice Generator.

Customer selection, invoice creation and dashboard statistics work.

Everything already built must continue working.

Return complete updated files only.

Do not return snippets.

Update only:

- dashboard.html
- dashboard.js
- auth.css

GENERAL GOAL

Replace the Invoice History placeholder with a complete list of saved invoices.

DASHBOARD.JS

Retrieve only invoices available to the signed-in user.

Order them by created_at with the newest first.

Display one card for each invoice.

Every card must show:

- Invoice number
- Customer name
- Issue date
- Due date
- Status
- Final total with currency

Add a View Invoice button linking to:

invoice-details.html?id=INVOICE_ID

Pass only the invoice ID in the address.

Create separate loading, empty, error and content areas.

Display only the correct area.

If no invoice exists, display:

You haven't created any invoices yet. Create your first invoice using the form above.

Refresh the history after an invoice is saved, edited or deleted.

Do not create duplicate cards during refresh.

Display database information safely.

Do not insert untrusted values using unsafe HTML.

AUTH.CSS

Keep every existing style.

Style invoice cards and status labels.

Keep the history readable on small screens.

IMPORTANT

Continue relying on Row Level Security.

Do not retrieve another user's invoices.

Return complete files only.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return complete updated versions of:

- dashboard.html
- dashboard.js
- auth.css

The dashboard should now display a complete Invoice History.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Replace the complete contents of every updated file.

Save each file.

TEST YOUR WORK

Sign in and open the dashboard.

Confirm that:

- ☒ The loading state appears while invoices load.
- ☒ Every saved invoice appears once.
- ☒ Each card displays the invoice number, customer, dates, status and total.
- ☒ View Invoice contains the correct invoice ID.
- ☒ A new account sees the friendly empty state.
- ☒ A failed request displays the friendly error state.
- ☒ Saving another invoice refreshes the history without duplicate cards.
- ☒ Only the signed-in user's invoices appear.

CHECKPOINT

Before moving on, confirm that:

- ☒ Invoice History works correctly.

- ✓ Invoice cards display correctly.
- ✓ Loading, empty and error states work.
- ✓ Invoice privacy remains protected.

COMMON BEGINNER MISTAKES

- Some beginners retrieve every invoice from the database and then filter them inside JavaScript.
- Always request only the authenticated user's records.
- Doing so improves both security and performance.

BEHIND THE SCENES

- Each invoice card is created from one invoice returned by Supabase.
- JavaScript converts database information into a visual component that users can recognise and interact with immediately.

THINK LIKE A SOFTWARE DESIGNER

- Business software should present information in a way that reduces effort.
- A well-designed invoice history allows users to recognise invoices quickly and perform common actions without unnecessary navigation.

CODE-READING QUESTION

Open dashboard.js. Find one function that supports building the invoice history. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- build a professional Invoice History
- display invoice details cards
- generate invoice number
- add contact actions
- handle loading, empty and error states
- refresh displayed information automatically

CHAPTER 6

VIEWING INVOICES

LESSON 2

BUILDING THE INVOICE DETAILS PAGE

Estimated Time

90 minutes

WHAT YOU ARE BUILDING

The Invoice History now allows users to see every invoice at a glance.

In this lesson, you will build the complete Invoice Details page.

Instead of displaying only summary information, the details page will securely retrieve one invoice from Supabase and display every available detail in a clean, professional layout.

Users will also be able to call customers, send emails, and begin the invoice editing process from this page.

WHY THIS MATTERS

Invoice cards are useful for browsing information quickly, but businesses often need to see the complete invoice before making decisions.

A dedicated details page provides enough space to display contact information, addresses, business information, notes and record details without overcrowding the Invoice History.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Invoice History works correctly.
- ☒ View Invoice buttons appear.
- ☒ Invoice cards display correctly.
- ☒ Invoice IDs are passed through the page address.
- ☒ invoice-details.html already exists.

- ✓ invoice-details.js already exists.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Professional Invoice Generator.

Invoice History links to invoice-details.html.

Everything already built must continue working.

Return complete updated files only.

Do not return snippets.

Update only:

- invoice-details.html
- invoice-details.js
- auth.css

GENERAL GOAL

Build the complete protected invoice details page.

INVOICE-DETAILS.JS

Confirm that the user is signed in.

Read the invoice ID from:

invoice-details.html?id=INVOICE_ID

If no ID exists, show a friendly unavailable message and do not query Supabase.

Retrieve one invoice where:

- id matches the address
- user_id matches the signed-in user

Retrieve all invoice_items belonging to that invoice.

Order items by position.

Continue relying on Row Level Security.

Do not reveal whether another user owns an unavailable invoice.

INVOICE-DETAILS.HTML

Display:

- Business details
- Invoice number
- Issue date
- Due date
- Status
- Saved customer snapshot
- Every invoice item
- Subtotal
- Discount
- Tax
- Final total and currency
- Notes

Add:

- Edit Invoice
- Back to Dashboard
- Logout

Create separate loading, unavailable, error and content areas.

Display money values to two decimal places.

Display dates clearly.

Display database information safely.

AUTH.CSS

Keep every existing style.

Create a clean responsive document layout.

IMPORTANT

Do not create another Supabase client.

Do not retrieve another user's invoice.

Return complete files only.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return complete updated versions of:

- invoice-details.html
- invoice-details.js
- auth.css

The application should now support complete invoice details.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Replace the complete contents of every updated file.

Save each file.

TEST YOUR WORK

Open Invoice History.

Select View Invoice.

Confirm that:

- ☒ The correct invoice opens.
- ☒ Business and saved customer details appear.
- ☒ Invoice number, issue date, due date and status appear.

- ✓ Every saved item appears in the correct order.
 - ✓ Subtotal, discount, tax and final total are correct.
 - ✓ Edit Invoice opens `dashboard.html?edit=INVOICE_ID`.
 - ✓ Back to Dashboard and Logout work.
- Remove the ID from the address.
- A friendly unavailable message should appear.
- Try an invoice ID belonging to another account.
- No invoice information should appear.

CHECKPOINT

Before moving on, confirm that:

- ✓ Invoice details load correctly.
- ✓ Every item and trusted total appears.
- ✓ Authentication works.
- ✓ Invoice ownership remains protected.
- ✓ Edit Invoice works.

COMMON BEGINNER MISTAKES

- Some beginners retrieve every invoice before selecting one inside JavaScript.
- Always request only the invoice that the page needs.
- Doing so improves security, performance and keeps the application easier to maintain.

BEHIND THE SCENES

- The invoice details page demonstrates one of the most common patterns in business software.
- Instead of creating one HTML page for every invoice, the application uses one reusable page that loads different information depending on the invoice ID supplied.

THINK LIKE A SOFTWARE DESIGNER

Reusable pages make software easier to maintain.

Whenever many records share the same layout, build one dynamic page instead of creating many separate pages.

CODE-READING QUESTION

Open `invoice-details.js`. Find one function that supports building the invoice details page. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- build a complete invoice details page
- retrieve one authorised invoice
- retrieve and order related invoice items
- protect invoice privacy
- create one reusable details page

CHAPTER 6

VIEWING INVOICES

LESSON 3

TESTING THE INVOICE HISTORY

Estimated Time

45 minutes

WHAT YOU ARE BUILDING

In this lesson, you will thoroughly test the complete Invoice History and Invoice Details system.

Rather than adding new features, you will confirm that Invoice History and Invoice Details work correctly under different conditions.

Professional software development always includes careful testing before new features are added.

WHY THIS MATTERS

Many software problems appear only after several features begin working together.

Testing helps you discover those problems before users rely on the application.

This lesson will confirm that invoice privacy, Invoice History and the details page all function correctly.

BEFORE YOU CONTINUE

Prepare:

- Two separate user accounts.
- Several Draft invoices.
- Several Paid invoices.

Include invoices with and without email addresses.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Complete every test carefully.

Correct any problems before continuing.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No files are created or updated.

TEST YOUR WORK

Complete every test on the deployed HTTPS website.

INVOICE HISTORY

- ✓ Loading, empty, error and content states work.
- ✓ Every invoice card appears once.
- ✓ Invoice numbers, customers, dates, statuses and totals are correct.

INVOICE DETAILS

- ✓ The correct invoice and items load.
- ✓ Trusted totals match the saved database values.
- ✓ Missing notes display a friendly fallback.
- ✓ Edit Invoice uses the correct invoice ID.

SECURITY

- ✓ A signed-out visitor returns to login.html.
- ✓ Account A cannot open an invoice belonging to Account B.
- ✓ An unavailable invoice does not reveal another user's information.

RESPONSIVE LAYOUT

- ✓ Cards and details remain readable on a small screen.

CHECKPOINT

Before completing this chapter, confirm that:

- ✓ Invoice History works correctly.
- ✓ Invoice Details work correctly.
- ✓ Authentication works.
- ✓ Invoice privacy works.
- ✓ Responsive layouts work.

COMMON BEGINNER MISTAKES

Many beginners assume that software works because one successful test passes.

Professional software is tested under many different situations, including missing information, different users and unusual screen sizes.

BEHIND THE SCENES

Invoice privacy depends on several layers working together:

- Authentication
- Secure queries
- Row Level Security
- Careful interface design

Removing any one of these layers weakens the application.

THINK LIKE A SOFTWARE DESIGNER

Good software is not only built carefully.

It is also tested carefully.

Professional developers spend significant time confirming that completed features continue working as new features are added.

WHAT YOU LEARNED

In this lesson you learned how to:

- test complete software features
- verify invoice privacy
- verify responsive layouts
- test authentication
- confirm professional software quality

CHAPTER SUMMARY

Congratulations.

Users can now browse saved invoices and open one complete document inside the application.

Invoice History handles loading, empty and error states.

The protected details page displays the business information, customer snapshot, items and trusted totals without exposing another user's invoice.

CHAPTER MILESTONE

By the end of Chapter 6, you have successfully built:

- ✓ Invoice History
- ✓ Invoice cards
- ✓ View Invoice links
- ✓ Complete invoice details
- ✓ Ordered invoice items
- ✓ Loading state
- ✓ Empty state
- ✓ Error state
- ✓ Responsive layouts
- ✓ Invoice privacy protection

TRANSITION TO CHAPTER 7

Your application can now create, organise and display invoice information professionally.

The next step is making those invoices much easier to find.

As invoice databases grow, scrolling through hundreds of invoice cards quickly becomes inefficient.

In the next chapter, you will build intelligent search, filtering and sorting capabilities that allow businesses to locate invoices within seconds.

CHAPTER 7

SEARCHING, FILTERING AND SORTING INVOICES

CHAPTER INTRODUCTION

Your Professional Invoice Generator can now create, store and display invoices professionally.

As the number of invoices grows, however, finding the right invoice becomes increasingly difficult.

Scrolling through hundreds of invoice cards is slow and frustrating.

Professional invoice management software solves this problem by giving users powerful tools for searching, filtering and organising information.

In this chapter, you will build a complete invoice discovery system that allows users to locate invoices within seconds.

The Invoice History will become far more useful without changing the invoice information stored in the database.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- Invoice History toolbar

- Search by invoice number
- Search by customer information
- Draft, sent, paid and overdue filters
- Date and total sorting
- Result information
- No-results state
- Clear Search and Filters
- Automatic result refresh

LESSON 1

BUILDING INVOICE SEARCH, FILTERING AND SORTING

Estimated Time

120 minutes

WHAT YOU ARE BUILDING

In this lesson, you will build a complete Invoice History toolbar.

Users will be able to:

- Search invoices while typing.
- Filter invoices by Status.
- Sort invoices.
- Clear all search controls instantly.

The Invoice History will update automatically without reloading the page.

WHY THIS MATTERS

Businesses often manage hundreds or thousands of invoices.

Without search and filtering, finding the correct invoice quickly becomes difficult.

Professional software should help users locate information in seconds instead of forcing them to browse long lists manually.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Invoice History works correctly.
- ☒ Invoice cards display correctly.
- ☒ Invoice details work correctly.

☒ Dashboard statistics work.

☒ Several invoices exist.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Professional Invoice Generator.

Invoice History and invoice details work.

Everything already built must continue working.

Return complete updated files only.

Do not return snippets.

Update only:

- dashboard.html
- dashboard.js
- auth.css

GENERAL GOAL

Make Invoice History searchable, filterable and sortable.

DASHBOARD.HTML

Add a toolbar above Invoice History.

Include:

- Search Invoices field
- Status filter
- Sort Invoices field
- Clear Search and Filters button
- Result information
- Separate no-results message

Search placeholder:

Search by invoice number or customer

Status options:

- All Statuses
- Draft
- Sent
- Paid
- Overdue

Sort options:

- Newest First
- Oldest First
- Due Date Soonest
- Highest Total
- Lowest Total

DASHBOARD.JS

Keep a reusable `allInvoices` array after invoices load.

Do not request the database again for every search.

Search while the user types across:

- Invoice number
- Customer name
- Customer company
- Customer email

Ignore letter case and extra spaces.

Filter by status.

Sort dates as dates and totals as numbers.

Apply search, filter and sorting together.

Update result information after every change.

Show the no-results message only when invoices exist but none match.

Clear Search and Filters must restore the default view.

AUTH.CSS

Keep every existing style.

Make the toolbar usable on desktop and small screens.

IMPORTANT

Return complete files only.

Do not remove invoice creation or statistics.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return complete updated versions of:

- dashboard.html
- dashboard.js
- auth.css

The Invoice History should now support professional searching, filtering and sorting.

SAVE YOUR FILES**BACK UP BEFORE REPLACING FILES**

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Replace the complete contents of every updated file.

Save each file.

TEST YOUR WORK

Confirm that:

- ☒ Searching by invoice number works.
- ☒ Searching by customer name works.

- ✓ Searching by customer company works.
- ✓ Searching ignores letter case and extra spaces.
- ✓ Draft, sent, paid and overdue filters work.
- ✓ Newest and oldest sorting work.
- ✓ Due Date Soonest sorts real dates correctly.
- ✓ Highest and Lowest Total sort numbers correctly.
- ✓ Search, filter and sorting work together.
- ✓ Result information updates.
- ✓ No-results and empty-history messages remain different.
- ✓ Clear Search and Filters restores the default history.
- ✓ Saving another invoice refreshes allInvoices and the visible results.

CHECKPOINT

Before moving on, confirm that:

- ✓ Search works.
- ✓ Status filtering works.
- ✓ Date and total sorting work.
- ✓ No-results and result information work.
- ✓ Invoice History refreshes correctly.

COMMON BEGINNER MISTAKES

- Many beginners request invoices from the database every time the user presses a key.
- The application already has the invoice information.
- Searching the existing invoice array is much faster and provides a smoother user experience.

BEHIND THE SCENES

- Invoice History retrieves the authorised invoices once.

JavaScript then searches, filters and sorts the reusable array in the browser.

This keeps the controls fast and avoids an unnecessary database request after every key press.

THINK LIKE A SOFTWARE DESIGNER

Good software does not make users adapt to the data.

Instead, it adapts the data to the way users naturally search for information.

CODE-READING QUESTION

Open dashboard.js. Find one function that supports building invoice search, filtering and sorting. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- build live invoice search
- filter invoices
- sort invoices
- improve software performance
- create a professional invoice discovery experience

CHAPTER 7

SEARCHING, FILTERING AND SORTING INVOICES

LESSON 2

TESTING INVOICE SEARCH

Estimated Time

45 minutes

WHAT YOU ARE BUILDING

In this lesson, you will test the complete invoice discovery system.

Rather than adding new functionality, you will confirm that searching, filtering and sorting continue working correctly under different situations.

WHY THIS MATTERS

Search features often work correctly when tested individually but fail when several controls are combined.

Professional testing ensures that every control behaves predictably.

BEFORE YOU CONTINUE

Prepare:

- Several Draft invoices
- Several Paid invoices
- Invoices with different names
- Invoices with different creation dates

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Complete every test carefully before continuing.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No files are created or updated.

TEST YOUR WORK

Confirm that:

- ☒ Searching by invoice number works.
- ☒ Searching by customer name works.
- ☒ Searching by customer company works.
- ☒ Searching ignores letter case and extra spaces.
- ☒ Draft, sent, paid and overdue filters work.
- ☒ Newest and oldest sorting work.
- ☒ Due Date Soonest sorts real dates correctly.
- ☒ Highest and Lowest Total sort numbers correctly.
- ☒ Search, filter and sorting work together.
- ☒ Result information updates.
- ☒ No-results and empty-history messages remain different.
- ☒ Clear Search and Filters restores the default history.
- ☒ Saving another invoice refreshes allInvoices and the visible results.

CHECKPOINT

Before completing this chapter, confirm that:

- ☒ Every Invoice History control works.

- ✓ Combined controls work correctly.
- ✓ Invoices remain protected.
- ✓ Invoice cards never duplicate.

COMMON BEGINNER MISTAKES

Some developers test every control separately but never test them together.

Professional software should behave correctly regardless of how many controls the user combines.

BEHIND THE SCENES

Every Invoice History update follows the same sequence:

Retrieve invoices once.

Apply search.

Apply filtering.

Apply sorting.

Display the final result.

Using one predictable process makes the application easier to maintain.

THINK LIKE A SOFTWARE DESIGNER

When several controls affect the same information, build one central workflow that coordinates them instead of allowing each control to work independently.

WHAT YOU LEARNED

In this lesson you learned how to:

- test combined software features
- verify invoice discovery
- test responsive interfaces
- validate professional application behaviour

CHAPTER SUMMARY

Congratulations.

Your Professional Invoice Generator now allows users to locate invoice information quickly using professional search, filtering and sorting tools.

Instead of browsing long invoice lists manually, users can now find the information they need within seconds.

These improvements significantly increase the usability of the software while keeping invoice information secure.

CHAPTER MILESTONE

By the end of Chapter 7, you have successfully built:

- ✓ Invoice History Toolbar
- ✓ Live Invoice Search
- ✓ Status Filtering
- ✓ Invoice Sorting
- ✓ Result Counter
- ✓ No-results State
- ✓ Clear Search and Filters
- ✓ Automatic Invoice History Refresh
- ✓ Professional Invoice Discovery

TRANSITION TO CHAPTER 8

Your application can now create, display and locate invoices efficiently.

In Chapter 8, you will build a secure editing workflow.

Users will update the selected customer, dates, status, items, discounts, tax and notes while Supabase protects ownership and recalculates every trusted total.

CHAPTER 8

EDITING INVOICES

CHAPTER INTRODUCTION

A saved invoice may need to change.

A customer may ask for another item.

A quantity or price may change.

The business may apply a different discount or tax rate.

The invoice status may also move from Draft to Sent or Paid.

In this chapter, you will allow the signed-in user to load one complete invoice, change its details and items, and save every change securely.

The application will recalculate all totals inside Supabase and will never allow one user to edit another user's invoice.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- Edit Invoice workflow
- Secure invoice retrieval
- Automatic form population

- Editing for all invoice items
- Add and Remove Item controls during editing
- Automatic recalculation
- Server-side total checking
- Secure database updates
- Cancel Edit action
- Automatic history and statistics refresh
- Invoice editing security tests

LESSON 1

BUILDING INVOICE EDITING

Estimated Time

120 minutes

WHAT YOU ARE BUILDING

In this lesson, you will build the complete invoice editing workflow.

Users will be able to open invoice details, select Edit Invoice, automatically load the existing invoice information into the dashboard form, update the information and save the changes securely.

The application will reuse the existing invoice form instead of creating a separate editing form.

WHY THIS MATTERS

Professional software avoids unnecessary duplication.

Instead of maintaining one form for creating invoices and another for editing invoices, the same form can perform both tasks.

This reduces maintenance, keeps the user experience consistent and ensures that invoice information always passes through the same validation rules.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Invoice details work correctly.
- ☒ Edit Invoice opens:
`dashboard.html?edit=INVOICE_ID`
- ☒ Invoice creation still works.
- ☒ Invoice History works.
- ☒ Dashboard statistics work.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Professional Invoice Generator.

Creating, saving and viewing complete invoices works.

Everything already built must continue working.

Return complete files only.

Do not return snippets.

Create or update only:

- update-invoice-function.sql
- dashboard.html
- dashboard.js
- invoice-details.js
- auth.css

GENERAL GOAL

Allow the signed-in user to edit one complete invoice and all its items securely.

UPDATE-INVOICE-FUNCTION.SQL

Create one complete PostgreSQL function named:

update_invoice_with_items

Use the normal authenticated caller and Row Level Security.

The function must:

- Require auth.uid()
- Accept invoice ID, customer ID, invoice details and a JSON item array
- Confirm that the invoice belongs to auth.uid()
- Confirm that the selected customer belongs to auth.uid()
- Copy the selected customer details into the invoice snapshot fields
- Validate issue date and due date

- Validate at least one item
 - Validate every description, quantity and unit price
 - Recalculate line totals, subtotal, discount, tax and final total in Supabase
 - Update the invoice
 - Replace its items with the new complete item list
 - Complete every change as one transaction
 - Leave the original invoice unchanged when any part fails
 - Update updated_at without changing created_at
 - Keep the existing invoice number unchanged
- Restrict execution to authenticated users.

DASHBOARD.JS

- Read the edit ID from:
- dashboard.html?edit=INVOICE_ID
- Load only the signed-in user's invoice and items.
- Fill the existing form and select the saved customer.
- Do not create a second form.
- Allow customer, dates, status, items, discount, tax and notes to change.
- Change Save Invoice to Update Invoice in edit mode.
- Add Cancel Edit.
- Call update_invoice_with_items after complete validation.
- Show friendly progress, success and error messages.
- Return to create mode after success.
- Refresh history and statistics.

IMPORTANT

- Do not create another Supabase client.
- Do not trust browser totals.
- Do not allow one user to edit another user's invoice.
- Return all five complete files.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return complete updated versions of:

- dashboard.html
- dashboard.js
- invoice-details.js
- auth.css

The application should now support complete invoice editing.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Replace the complete contents of every updated file.

Save each file.

TEST YOUR WORK

Open one saved invoice.

Select:

Edit Invoice

Confirm that:

- ☒ Edit mode uses the existing invoice form.

☒ The correct customer is selected.

☒ Every invoice field and item loads.

Change the customer, due date, status, one item, discount, tax and notes.

Select:

Update Invoice

Confirm that:

☒ Complete validation runs.

☒ The invoice number does not change.

☒ Trusted totals are recalculated in Supabase.

☒ The customer snapshot updates to the selected customer.

☒ created_at remains unchanged.

☒ updated_at changes.

☒ History, details and statistics refresh.

Cancel a second edit.

The saved invoice should remain unchanged.

Attempt to edit another account's invoice.

Access must be denied and no update must occur.

CHECKPOINT

Before moving on, confirm that:

☒ Edit mode loads the complete invoice.

☒ Customer and item changes work.

☒ Trusted totals are recalculated.

☒ The invoice number and created_at remain unchanged.

☒ The update is transactional and secure.

COMMON BEGINNER MISTAKES

Some beginners build a completely separate editing form.

Professional applications often reuse the same form for both creating and editing records.

This reduces duplicated code and keeps behaviour consistent.

BEHIND THE SCENES

The invoice form now supports two modes.

When no invoice ID exists, it creates a new invoice.

When an edit invoice ID exists, it updates the existing invoice.

The same interface now supports two different business workflows.

THINK LIKE A SOFTWARE DESIGNER

Whenever two features perform nearly the same task, look for opportunities to reuse existing components instead of creating new ones.

Reusable software is easier to maintain and usually contains fewer bugs.

CODE-READING QUESTION

Open `dashboard.js`. Find one function that supports building invoice editing. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- build complete invoice editing
- reuse existing software components
- update invoices securely
- protect invoice ownership
- refresh business information automatically

CHAPTER 8

EDITING INVOICES

LESSON 2

TESTING INVOICE EDITING

Estimated Time

50 minutes

WHAT YOU ARE BUILDING

In this lesson, you will test the complete invoice editing workflow.

You will confirm that authorised changes succeed, trusted totals remain correct, cancelled changes are not saved, and another account cannot update the invoice.

WHY THIS MATTERS

Editing changes existing business information.

Testing helps ensure that authorised users can update their own invoices without accidentally affecting another user's data.

BEFORE YOU CONTINUE

Prepare:

- Two user accounts
- Two saved customers
- Several invoices containing more than one item
- Invoices with different statuses, discounts and tax values

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Complete every test carefully.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No files are created or updated.

TEST YOUR WORK

Open one saved invoice.

Select:

Edit Invoice

Confirm that:

☒ Edit mode uses the existing invoice form.

☒ The correct customer is selected.

☒ Every invoice field and item loads.

Change the customer, due date, status, one item, discount, tax and notes.

Select:

Update Invoice

Confirm that:

☒ Complete validation runs.

☒ The invoice number does not change.

☒ Trusted totals are recalculated in Supabase.

☒ The customer snapshot updates to the selected customer.

☒ created_at remains unchanged.

☒ updated_at changes.

☒ History, details and statistics refresh.

Cancel a second edit.

The saved invoice should remain unchanged.

Attempt to edit another account's invoice.

Access must be denied and no update must occur.

CHECKPOINT

Before completing this chapter, confirm that:

- ✓ Complete invoice editing works.
- ✓ Cancel Edit leaves the saved invoice unchanged.
- ✓ Trusted totals remain correct.
- ✓ Invoice ownership remains protected.
- ✓ History, details and statistics refresh.

COMMON BEGINNER MISTAKES

A common mistake is updating the invoice first and replacing its items afterwards using unrelated requests.

If the item update fails, the document becomes inconsistent.

The update function completes every change as one database transaction.

BEHIND THE SCENES

Invoice editing follows the same sequence used during invoice creation.

The update function replaces the authorised invoice and its items as one transaction.

Every other quality and security check remains the same.

THINK LIKE A SOFTWARE DESIGNER

Consistency makes software easier to understand.

Whenever possible, editing should follow the same rules as creating new records.

WHAT YOU LEARNED

In this lesson you learned how to:

- test complete invoice editing
- verify trusted recalculation
- verify transactional updates
- test cancelled changes
- protect invoice ownership

CHAPTER SUMMARY

Congratulations.

Authorised users can now edit one complete invoice and all its items.

The application reuses the existing form, preserves the invoice number and original creation date, updates the customer snapshot, and recalculates trusted totals inside Supabase.

Unauthorised users cannot load or update the invoice.

CHAPTER MILESTONE

By the end of Chapter 8, you have successfully built:

- ✓ Invoice edit mode
- ✓ Secure invoice retrieval
- ✓ Existing form population
- ✓ Customer and item editing
- ✓ Trusted total recalculation
- ✓ Transactional invoice update
- ✓ Preserved invoice number and created date
- ✓ Cancel Edit
- ✓ Automatic history and statistics refresh
- ✓ Two-account editing protection

TRANSITION TO CHAPTER 9

Your application can now create, view, find and edit invoices.

In Chapter 9, you will produce a clean printable A4 document and add a confirmed deletion workflow that removes the invoice and its related items securely.

CHAPTER 9

PRINTING AND DELETING INVOICES

CHAPTER INTRODUCTION

Businesses need to share invoices in a clear, professional format.

They also need a safe way to remove invoices that are no longer required.

In this chapter, you will first create a clean A4 invoice document that can be printed from the browser.

The printed page will show the business details, customer details, items, discount, tax and final total.

You will then add secure invoice deletion.

Because deletion cannot easily be undone, the application will ask for confirmation and will delete only an invoice that belongs to the signed-in user.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- Professional A4 invoice document
- Complete invoice item table
- Print Invoice button
- Print-only styles

- Hidden screen controls during printing
- Delete Invoice action
- Delete confirmation
- Secure invoice deletion
- Automatic invoice item deletion
- Invoice History refresh
- Dashboard statistics refresh
- Printing and deletion testing

LESSON 1

BUILDING THE PRINTABLE INVOICE DOCUMENT

Estimated Time

90 minutes

WHAT YOU ARE BUILDING

In this lesson, you will build a professional invoice document that can be printed from the browser.

The document will show the complete invoice in a clean A4 layout.

Screen controls will disappear during printing so the customer sees only the invoice itself.

WHY THIS MATTERS

An invoice may look clear on a computer screen but print badly on paper.

Navigation menus, buttons and narrow layouts do not belong on a professional document.

Print styles allow the same invoice page to become a clean business document when the user selects Print Invoice.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Invoice details load correctly.
- ☒ Every invoice item appears.
- ☒ Subtotal, discount, tax and final total are correct.
- ☒ The invoice belongs to the signed-in user.
- ☒ Row Level Security remains enabled.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Professional Invoice Generator.

Users can open a complete saved invoice.

Everything already built must continue working.

Return complete updated files only.

Do not return snippets.

Update only:

- invoice-details.html
- invoice-details.js
- auth.css

GENERAL GOAL

Build a professional A4 invoice document that the user can print.

INVOICE-DETAILS.HTML

Keep every feature that already works.

Add a Print Invoice button.

Create an invoice document area containing:

- Business details
- Invoice number
- Issue date
- Due date
- Status
- Saved customer details
- Every invoice item
- Subtotal
- Discount

- Tax
- Final total and currency
- Notes

INVOICE-DETAILS.JS

- Load the invoice only when it belongs to the signed-in user.
- Load and display items in their saved position.
- Display money values to two decimal places.
- Use the browser print window when Print Invoice is selected.

AUTH.CSS

- Add print styles using:
 - @media print
- Use a clean A4 layout.
- Hide navigation, editing controls, buttons and screen-only messages when printing.
- Keep the invoice document visible.
- Prevent item rows from breaking awkwardly.

IMPORTANT

- Do not create another Supabase client.
- Do not allow one user to print another user's invoice.
- Return complete files only.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return complete updated versions of:

- invoice-details.html
- invoice-details.js
- dashboard.js
- auth.css

The application should now support secure invoice deletion.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Replace the complete contents of every updated file.

Save each file.

TEST YOUR WORK

Open a saved invoice that contains several items.

Confirm that:

- ☒ Business details appear.
- ☒ Customer details appear.
- ☒ Invoice number and dates appear.
- ☒ Every invoice item appears.
- ☒ Subtotal, discount, tax and final total are correct.

Select:

Print Invoice

Confirm that:

- ☒ The browser print window opens.
- ☒ The invoice fits a clean A4 layout.
- ☒ Navigation and buttons are hidden.
- ☒ No important information is cut off.

Cancel the print window.

Confirm that the normal screen layout still works.

CHECKPOINT

Before moving on, confirm that:

- ☒ Confirmation works.
- ☒ Invoice deletion works.
- ☒ Dashboard refreshes.
- ☒ Statistics refresh.
- ☒ Invoice ownership remains protected.

COMMON BEGINNER MISTAKES

Some beginners delete records immediately after the Delete button is selected.

Professional software always asks the user to confirm destructive actions.

Another common mistake is deleting records using only the invoice ID.

Every deletion should also verify the authenticated user's ownership.

BEHIND THE SCENES

Deleting an invoice follows the same security principles used throughout this workbook.

The application confirms the user's intention, verifies authentication, checks invoice ownership and then performs the database operation.

Each layer reduces the risk of accidental or unauthorised deletion.

THINK LIKE A SOFTWARE DESIGNER

The more serious the action, the more carefully it should be designed.

Users should always understand the consequences before permanently removing important information.

CODE-READING QUESTION

Open invoice-details.js. Find one function that supports building the printable invoice document. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- build secure invoice deletion
- protect invoice ownership
- confirm destructive actions
- refresh application data automatically

CHAPTER 9

PRINTING AND DELETING INVOICES

LESSON 2

BUILDING AND TESTING SECURE INVOICE DELETION

Estimated Time

40 minutes

WHAT YOU ARE BUILDING

In this lesson, you will build and test secure invoice deletion.

The user will be able to delete one invoice after confirming the action.

The related invoice items will be removed automatically.

The Invoice History and dashboard statistics will then refresh.

WHY THIS MATTERS

Deleting information is permanent.

A professional application should never remove an invoice because of one accidental click.

Confirmation, authenticated ownership checks and Row Level Security work together to protect the user's information.

BEFORE YOU CONTINUE

Prepare:

- Two authenticated user accounts.
- Several invoices.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Professional Invoice Generator.

Printing, editing and viewing invoices already work.

Everything built so far must continue working exactly as before.

Continue using the shared supabaseClient from supabase-config.js.

Return complete updated files only.

Do not return snippets or partial code.

Update only:

- invoice-details.html
- invoice-details.js
- dashboard.js
- auth.css

GENERAL GOAL

Build the complete secure invoice deletion workflow.

Add a Delete Invoice button to the invoice details page.

Ask for confirmation before deleting.

If the user cancels, stop immediately.

If the user confirms:

- Confirm the user is signed in
- Delete only the invoice with the current ID
- Include the signed-in user's ID in the delete request
- Continue relying on Row Level Security
- Allow ON DELETE CASCADE to remove the related invoice items

After a successful deletion:

- Display a friendly success message
- Return to dashboard.html

- Refresh invoice history
 - Refresh dashboard statistics
- Handle errors with a friendly message.
- Use `console.error()` for technical details.
- Do not display raw database errors.
- Do not allow one user to delete another user's invoice.
- Return complete updated files only.
- Do not return snippets or partial code.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return complete updated versions of:

- `invoice-details.html`
- `invoice-details.js`
- `dashboard.js`
- `auth.css`

The application should now support secure invoice deletion.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Replace the complete contents of every updated file.

Save each file.

Do not change the file names.

TEST YOUR WORK

Confirm that:

- ☒ Delete Invoice appears.
- ☒ Confirmation dialog appears.
- ☒ Cancelling deletion leaves the invoice unchanged.
- ☒ Confirming deletion removes the invoice.
- ☒ Dashboard statistics update.
- ☒ Invoice History refreshes.
- ☒ Deleted invoice details no longer opens.

Attempt to delete another user's invoice.

Confirm that:

- ☒ Deletion is prevented.
- ☒ Invoice information remains protected.

Delete several invoices one after another.

Confirm that:

- ☒ Invoice History remains accurate.
- ☒ Dashboard statistics remain accurate.
- ☒ No unexpected errors occur.

Test the application on a smaller screen.

Confirm that:

- ☒ Delete Invoice remains accessible.
- ☒ Confirmation remains understandable.

CHECKPOINT

Before completing this chapter, confirm that:

- ✓ Invoice deletion works.
- ✓ Confirmation works.
- ✓ Dashboard refreshes.
- ✓ Invoice ownership remains protected.
- ✓ Dashboard statistics remain accurate.

COMMON BEGINNER MISTAKES

Many beginners test only successful deletion.

Professional testing also confirms that cancelled deletions leave invoice information untouched and that another user's records remain protected.

BEHIND THE SCENES

Deleting an invoice affects several parts of the application.

Removing the database record is only one step.

The Invoice History, dashboard statistics and invoice details pages must also respond correctly after the deletion.

THINK LIKE A SOFTWARE DESIGNER

Every destructive action should be tested more thoroughly than ordinary features.

Protecting user information is one of the primary responsibilities of professional software.

CODE-READING QUESTION

Open invoice-details.js. Find one function that supports building and testing secure invoice deletion. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- test secure deletion
- verify ownership protection
- verify dashboard updates
- verify application consistency

CHAPTER SUMMARY

Congratulations.

Your application can now present each invoice as a professional A4 document and open the browser print window.

It can also delete an authorised invoice after confirmation.

The database relationship removes the related invoice items automatically while Row Level Security protects ownership.

CHAPTER MILESTONE

By the end of Chapter 9, you have successfully built:

- ✓ Professional printable invoice
- ✓ Complete item table
- ✓ A4 print styles
- ✓ Hidden screen controls during print
- ✓ Delete confirmation
- ✓ Secure invoice deletion
- ✓ Cascading item deletion
- ✓ History and statistics refresh
- ✓ Printing and deletion security tests

TRANSITION TO CHAPTER 10

Your Professional Invoice Generator is now functionally complete.

The final chapter focuses on ensuring that every feature works together reliably.

You will perform a complete end-to-end review of the application, verify every business workflow and prepare the software for deployment.

CHAPTER 10

FINAL APPLICATION REVIEW

CHAPTER INTRODUCTION

Congratulations.

You have built a complete Professional Invoice Generator.

The application now supports user accounts, saved customers, invoice creation, trusted calculations, history, search, editing, printing and deletion.

Before publishing, professional developers test the complete journey instead of checking isolated features only.

In this chapter, you will perform that final end-to-end review and confirm that different user accounts remain private.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will:

- Perform complete end-to-end testing
- Verify invoice security
- Verify Row Level Security
- Test responsive layouts
- Test application usability
- Prepare the application for deployment

LESSON 1

COMPLETE SYSTEM TESTING

Estimated Time

90 minutes

WHAT YOU ARE BUILDING

In this lesson, you will perform a complete review of your Professional Invoice Generator.

Instead of testing one feature at a time, you will test the application exactly as a real business user would.

By the end of this lesson, you should have complete confidence that every feature works correctly before publishing your software.

WHY THIS MATTERS

Software is only valuable when all its individual parts work together reliably.

Professional software development does not end when the last feature is written.

It ends when the complete application has been tested thoroughly.

BEFORE YOU CONTINUE

Prepare:

- Two user accounts
- Several Draft invoices
- Several Paid invoices
- Invoices with and without email addresses
- Invoices with complete and incomplete optional information

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Complete every test carefully.

If you discover a problem, correct it before deploying your application.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No files are created or updated during this lesson.

TEST YOUR WORK

Complete the following final tests on the deployed HTTPS website.

USER ACCOUNTS

- ☒ Registration, verification, login and logout work.
- ☒ Protected pages reject signed-out visitors.

CUSTOMERS

- ☒ A customer saves with the authenticated user's ID.
- ☒ The customer selection list and preview work.
- ☒ Account A cannot access Account B's customers.

INVOICE CREATION

- ☒ Customer selection is required.

- ✓ Multiple items, discounts and tax calculate correctly.
- ✓ Supabase recalculates trusted totals.
- ✓ The invoice and all items save as one complete action.

INVOICE HISTORY AND DETAILS

- ✓ Loading, empty, error and content states work.
- ✓ The correct invoice, snapshot, items and totals appear.

SEARCH, FILTER AND SORT

- ✓ Invoice number and customer search work.
- ✓ Status filters work.
- ✓ Date and total sorting work.

INVOICE EDITING

- ✓ Customer, fields and items update securely.
- ✓ The invoice number and created_at remain unchanged.

PRINTING

- ✓ The complete document prints cleanly on A4.
- ✓ Screen controls are hidden from print.

DELETION

- ✓ Cancelling leaves the invoice unchanged.
- ✓ Confirming deletes the invoice and cascades to its items.

SECURITY

- ✓ Account A cannot view, edit, print or delete Account B's invoice.
- ✓ Row Level Security remains enabled on all three tables.

RESPONSIVE DESIGN

- ✓ Customer, invoice, history, details and toolbar layouts remain usable on a small screen.

CONSOLE

- ✓ No unexpected red errors appear during the complete workflow.

CHECKPOINT

Before publishing your software, confirm that:

- ✓ Every feature works.
- ✓ Authentication works.
- ✓ Invoice ownership is protected.
- ✓ Dashboard statistics remain accurate.
- ✓ Invoice History remains accurate.
- ✓ Invoice details remain accurate.
- ✓ Responsive layouts work correctly.
- ✓ No unexpected JavaScript errors appear.

COMMON BEGINNER MISTAKES

Some developers stop testing after the application appears to work once.

Professional developers continue testing until they are confident that users can complete every important workflow successfully.

Another common mistake is testing only with one user account.

Always test with multiple accounts to confirm that invoice privacy remains protected.

BEHIND THE SCENES

Every capability depends on the others.

Authentication identifies the user.

The customer list supplies the invoice form.

Secure database functions save and update complete invoices.

Invoice History opens the protected details page.

Printing and deletion use that same authorised invoice.

Dashboard statistics reflect every successful change.

End-to-end testing confirms that the complete journey remains reliable.

THINK LIKE A SOFTWARE DESIGNER

Software development is not simply about writing code.

It is about delivering reliable software that people can trust.

Every hour spent testing before deployment saves many hours of fixing problems later.

WHAT YOU LEARNED

In this lesson you learned how to:

- perform complete application testing
- verify invoice security
- verify software reliability
- prepare software for deployment

FINAL TESTING

Before considering this workbook complete, perform one final review.

Create a completely new user account.

Begin with an empty invoice database.

Work through the application exactly as a first-time business user would.

Complete the following journey:

- ✓ Register
- ✓ Verify email
- ✓ Sign in

- ✓ Create an invoice with several items
- ✓ Check the subtotal
- ✓ Apply a discount
- ✓ Apply tax
- ✓ Save the invoice
- ✓ Open the invoice details
- ✓ Search, filter and sort invoices
- ✓ Edit one invoice
- ✓ Print one invoice
- ✓ Delete one invoice
- ✓ Sign out
- ✓ Sign in again

Confirm that every remaining invoice still exists and every dashboard statistic is correct.

If every step succeeds without unexpected errors, your Professional Invoice Generator is ready to be published.

GOING LIVE

Your Professional Invoice Generator is ready for its final deployment.

1.

Save every complete project file.

2.

Open:

<https://app.netlify.com/drop>

3.

Drag the complete Professional Invoice Generator folder into the deployment area.

If you already created the Netlify website during authentication testing, deploy the updated folder to that same website.

4.

Wait for the deployment to finish.

5.

Open the HTTPS website address supplied by Netlify.

6.

Confirm that the Supabase Site URL and Redirect URLs still use this exact website address.

7.

Repeat the complete live workflow:

- Registration and email verification
- Customer saving and selection
- Invoice creation and trusted calculations
- History, search and invoice details
- Editing
- Printing
- Deletion
- Two-account privacy

Only share the application after every important live test succeeds.

PORTFOLIO DESCRIPTION

Congratulations.

You have successfully built a fully functional Professional Invoice Generator.

Your application demonstrates practical software development skills including:

- User authentication
- Database design
- Secure invoice management
- Multiple invoice items
- Automatic totals, discounts and tax
- Create, read, update and delete operations
- Search and filtering
- Professional print layouts

- Responsive design
- Database security using Row Level Security

This project is an excellent addition to your software development portfolio because it solves a genuine business problem and demonstrates that you can build secure, data-driven applications using Artificial Intelligence.

REFLECTION QUESTIONS

Before moving to the next workbook, take a few minutes to think about what you have learned.

1.

Which part of this project helped you understand software development better?

2.

How did AI help you build this application more efficiently?

3.

Which software development concepts now make more sense than they did before beginning this workbook?

4.

If you were building this application for a real business, what additional features would you consider adding?

5.

What part of the project challenged you the most, and how did you solve it?

Writing down your answers will help reinforce what you have learned.

EXTENSION CHALLENGES

If you would like to improve your Professional Invoice Generator further, consider adding some of these features.

- Business logo upload
- Customer list
- Product and service list
- Invoice approval
- Invoice overdue reminders
- Email delivery
- PDF download
- More currencies
- Convert a paid invoice into a receipt
- Invoice activity log

Do not attempt to build every feature immediately.

Choose one feature at a time and treat it as a separate software project.

NEXT WORKBOOK

Excellent work.

You have completed Workbook 04 of the Prompt to Profit™ Software Workbook Series.

You have built software that allows a business to create, calculate, save, find, edit, print and delete professional invoices.

You have also protected every invoice using user authentication and Row Level Security.

The next project in the series is Workbook 05 — Appointment Booking System.

For now, take time to review what you have built and make sure every important feature works correctly.

Congratulations once again on completing your Professional Invoice Generator.